

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

```
├── src/
│   ├── chess/
│   │   ├── ai.js (23913 B / 4254b6b11040)
│   │   ├── app.js (37627 B / 37cf92152687)
│   │   ├── chess-core.js (37528 B / b313d1359659)
│   │   └── chess-runtime.js (540 B / 958b2454f794)
│   ├── styles/
│   │   ├── main.css (13601 B / 8946cbef0b66)
│   │   └── reset.css (560 B / 670e71371181)
│   └── main.js (83 B / b963e36e3c82)
└── index.html (7693 B / ef5c49ceb019)
```

==== index.html (7693 B / ef5c49ceb019) ====

```
1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>TS Chess</title>
7 |     <link rel="stylesheet" href="/src/styles/reset.css" />
8 |     <link rel="stylesheet" href="/src/styles/main.css" />
9 |   </head>
10 |   <body>
11 |     <main id="chess-app" class="chess-app">
12 |       <aside class="control-panel" aria-label="게임 컨트롤">
13 |         <section class="control-group" aria-label="모드">
14 |           <span class="control-label">모드</span>
15 |           <div class="segmented-control" role="group" aria-label="플레이 모드">
16 |             <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>
17 |             <button class="segmented-button" type="button" data-mode="ai">AI 상대</button>
18 |           </div>
19 |         </section>
20 |
21 |         <section id="color-controls" class="control-group" aria-label="진영">
22 |           <span class="control-label">진영</span>
23 |           <div class="segmented-control" role="group" aria-label="인간 진영">
24 |             <button class="segmented-button" type="button" data-color="w">백</button>
25 |             <button class="segmented-button" type="button" data-color="b">흑</button>
26 |           </div>
27 |         </section>
28 |
29 |         <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">
30 |           <span class="control-label">AI 설정</span>
31 |           <div
32 |             class="segmented-control segmented-control-three"
33 |             role="group"
34 |             aria-label="AI 프리셋"
35 |           >
36 |             <button class="segmented-button" type="button" data-preset="easy">쉬움</button>
37 |             <button class="segmented-button" type="button" data-preset="normal">보통</button>
38 |             <button class="segmented-button" type="button" data-preset="hard">어려움</button>
39 |           </div>
40 |           <label class="range-control">
41 |             <span>최대 깊이<strong id="ai-depth-value"></strong></span>
42 |             <input id="ai-depth-input" type="range" min="1" max="5" step="1" />
43 |           </label>
44 |           <label class="range-control">
45 |             <span>시간 제한<strong id="ai-time-value"></strong></span>
46 |             <input id="ai-time-input" type="range" min="500" max="5000" step="250" />
47 |           </label>
48 |         </section>
49 |
50 |         <section class="control-group clock-settings" aria-label="시간 설정">
51 |           <span class="control-label">시간</span>
52 |           <div class="clock-preset-control" role="group" aria-label="시간 프리셋">
53 |             <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
```

```

54 |         <button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
55 |         <button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
56 |         <button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
57 |         <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
58 |     </div>
59 | </section>
60 |
61 |     <section class="action-row" aria-label="게임·동작">
62 |         <button id="new-game-button" class="primary-button" type="button">새 게임</button>
63 |         <button id="undo-button" class="ghost-button" type="button">되돌리기</button>
64 |     </section>
65 | </aside>
66 |
67 |     <section class="board-stage" aria-label="체스판">
68 |         <div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
69 |     </section>
70 |
71 |     <aside class="clock-panel" aria-label="체스·타이머">
72 |         <div id="clock-display" class="clock-display">
73 |             <div id="white-clock" class="clock-card">
74 |                 <span class="clock-side">백</span>
75 |                 <strong id="white-clock-time" class="clock-time"></strong>
76 |             </div>
77 |             <div id="black-clock" class="clock-card">
78 |                 <span class="clock-side">흑</span>
79 |                 <strong id="black-clock-time" class="clock-time"></strong>
80 |             </div>
81 |         </div>
82 |     </aside>
83 |
84 |     <aside class="info-panel" aria-label="게임·정보">
85 |         <section class="status-panel" aria-live="polite">
86 |             <span class="control-label">상태</span>
87 |             <strong id="game-status" class="game-status"></strong>
88 |             <dl class="status-grid">
89 |                 <div>
90 |                     <dt>차례</dt>
91 |                     <dd id="turn-meta"></dd>
92 |                 </div>
93 |                 <div>
94 |                     <dt>모드</dt>
95 |                     <dd id="mode-meta"></dd>
96 |                 </div>
97 |                 <div>
98 |                     <dt>수순</dt>
99 |                     <dd id="move-meta"></dd>
100 |                 </div>
101 |             </dl>
102 |         </section>
103 |
104 |         <section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI·정보" hidden>
105 |             <div class="ai-candidate-header">
106 |                 <span class="control-label">AI·정보</span>
107 |                 <span id="ai-candidate-meta" class="ai-candidate-meta"></span>
108 |             </div>
109 |             <dl class="ai-info-grid">
110 |                 <div>
111 |                     <dt>상태</dt>
112 |                     <dd id="ai-info-status"></dd>
113 |                 </div>
114 |                 <div>
115 |                     <dt>난이도</dt>
116 |                     <dd id="ai-info-preset"></dd>
117 |                 </div>
118 |                 <div>
119 |                     <dt>인간</dt>
120 |                     <dd id="ai-info-human"></dd>
121 |                 </div>
122 |                 <div>
123 |                     <dt>depth</dt>
124 |                     <dd id="ai-info-depth"></dd>

```

```

125 |         </div>
126 |         <div>
127 |             <dt>nodes</dt>
128 |             <dd id="ai-info-nodes"></dd>
129 |         </div>
130 |         <div>
131 |             <dt>score</dt>
132 |             <dd id="ai-info-score"></dd>
133 |         </div>
134 |     </dl>
135 |     <div class="ai-candidate-table" role="table" aria-label="AI 후보 수 평가">
136 |         <div class="ai-candidate-row ai-candidate-head" role="row">
137 |             <span role="columnheader">SAN</span>
138 |             <span role="columnheader">cp</span>
139 |             <span role="columnheader">weight</span>
140 |             <span role="columnheader">prob</span>
141 |         </div>
142 |         <div id="ai-candidate-list" class="ai-candidate-list"></div>
143 |     </div>
144 | </section>
145 |
146 |     <section class="history-panel" aria-label="기보">
147 |         <div class="history-header">
148 |             <span class="control-label">기보</span>
149 |         </div>
150 |         <div id="move-list" class="move-list"></div>
151 |     </section>
152 | </aside>
153 |
154 |     <div id="promotion-overlay" class="promotion-overlay" hidden>
155 |         <div
156 |             class="promotion-dialog"
157 |             role="dialog"
158 |             aria-modal="true"
159 |             aria-labelledby="promotion-title"
160 |             >
161 |             <h2 id="promotion-title">승격</h2>
162 |             <div
163 |                 id="promotion-choices"
164 |                 class="promotion-choices"
165 |                 role="group"
166 |                 aria-label="승격 기물"
167 |             ></div>
168 |         </div>
169 |     </div>
170 | </main>
171 | <script id="ai-worker-source-1" type="text/plain">
172 |     !(function (ai_ts) {
173 |         'use strict';
174 |         const workerScope = globalThis;
175 |         workerScope.addEventListener('message', (event) => {
176 |             const response = ai_ts.searchBestMove(event.data, (progress) => {
177 |                 workerScope.postMessage(progress);
178 |             });
179 |             workerScope.postMessage({ ... response, kind: 'result' });
180 |         });
181 |     })(globalThis.__TSCSChess.ai);
182 | </script>
183 | <script src="/src/chess/chess-core.js" defer></script>
184 | <script src="/src/chess/chess-runtime.js" defer></script>
185 | <script src="/src/chess/ai.js" defer></script>
186 | <script src="/src/chess/app.js" defer></script>
187 | <script src="/src/main.js" defer></script>
188 | </body>
189 | </html>
190 |

```

==== src/chess/ai.js (23913 B / 4254b6b11040) ====

```

1 | function __TSCSChessCreateAi() {
2 |     var __TSCSChessAi = (function (exports, chessRuntime_ts) {
3 |         'use strict';

```

```

4 |     const HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
5 |     AI_PRESETS = {
6 |       easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },
7 |       normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
8 |       hard: {
9 |         preset: 'hard',
10 |         maxDepth: HARD_MAX_DEPTH,
11 |         timeLimitMs: 5e3,
12 |         randomness: 30,
13 |         quiescence: !0,
14 |       },
15 |     },
16 |     DEFAULT_AI_SETTINGS = { ... AI_PRESETS.normal },
17 |     PIECE_VALUES = { p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 },
18 |     STARTING_NON_PAWN_MATERIAL =
19 |       2 * (PIECE_VALUES.q + 2 * PIECE_VALUES.r + 2 * PIECE_VALUES.b + 2 * PIECE_VALUES.n),
20 |     OPENING_PHASE_END_MATERIAL = STARTING_NON_PAWN_MATERIAL / 2;
21 |     function normalizeAiSettings(settings) {
22 |       const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
23 |       return {
24 |         preset: settings.preset,
25 |         maxDepth:
26 |           ((depth = settings.maxDepth),
27 |           (preset = settings.preset),
28 |           'hard' === preset ? HARD_MAX_DEPTH : clamp(Math.round(depth), 1, 5)),
29 |         timeLimitMs: timeLimitMs,
30 |         randomness: 30,
31 |         quiescence: !0,
32 |       };
33 |       var depth, preset;
34 |     }
35 |     function searchRoot(chess, depth, ctx, previousBestMove) {
36 |       const candidates = [],
37 |         legalMoves = orderMoves(chess.moves({ verbose: !0 }), previousBestMove);
38 |       for (const move of legalMoves) {
39 |         (checkTime(ctx), chess.move(toMoveCommand(move)));
40 |         const score = -negamax(chess, depth - 1, -1e7, 1e7, 1, ctx);
41 |         (chess.undo(), candidates.push({ move: toMoveCommand(move), san: move.san, score: score }));
42 |       }
43 |       if (0 === candidates.length) throw new SearchTimeout();
44 |       const principal = (function selectBestRootMove(candidates) {
45 |         return candidates.reduce((currentBest, candidate) =>
46 |           candidate.score > currentBest.score ? candidate : currentBest,
47 |           {});
48 |       })(candidates),
49 |       selection = (function selectRootMove(candidates, settings, best) {
50 |         const level = settings.randomness / 30,
51 |           windowCp = 20 + 150 * level,
52 |           floorScore = best.score - windowCp,
53 |           ticketPower = 4 - 1.5 * level;
54 |         if (
55 |           (function shouldForceBestMove(settings, best) {
56 |             return (
57 |               0 === settings.randomness ||
58 |               (function isMateProtectedScore(score) {
59 |                 return Math.abs(score) >= 99e4;
60 |               })(best.score) ||
61 |               ('hard' === settings.preset && best.score < 0)
62 |             );
63 |           })(settings, best)
64 |         )
65 |         return {
66 |           selected: best,
67 |           debug: createDeterministicDebug(
68 |             candidates,
69 |             best,
70 |             best,
71 |             windowCp,
72 |             floorScore,
73 |             ticketPower,
74 |             ),

```

```

75 |         };
76 |         const ticketCandidates = (function selectTicketCandidates(candidates, settings, best) {
77 |             return 'hard' === settings.preset && best.score > 0
78 |             ? candidates.filter((candidate) => candidate.score > 0)
79 |             : candidates;
80 |         })(candidates, settings, best),
81 |         weightedCandidates = ticketCandidates
82 |             .map((candidate) => ({
83 |                 candidate: candidate,
84 |                 tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
85 |             }));
86 |         .filter(({ tickets: tickets }) => tickets > 0),
87 |         totalTickets = weightedCandidates.reduce(
88 |             (total, { tickets: tickets }) => total + tickets,
89 |             0,
90 |         );
91 |         if (totalTickets ≤ 0)
92 |             return {
93 |                 selected: best,
94 |                 debug: createDeterministicDebug(
95 |                     candidates,
96 |                     best,
97 |                     best,
98 |                     windowCp,
99 |                     floorScore,
100 |                     ticketPower,
101 |                 ),
102 |             };
103 |         let draw = Math.random() * totalTickets,
104 |             selected = best;
105 |         for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
106 |             if (((draw -= tickets), draw ≤ 0)) {
107 |                 selected = candidate;
108 |                 break;
109 |             }
110 |         return {
111 |             selected: selected,
112 |             debug: createWeightedDebug(
113 |                 candidates,
114 |                 best,
115 |                 selected,
116 |                 weightedCandidates,
117 |                 windowCp,
118 |                 floorScore,
119 |                 ticketPower,
120 |                 totalTickets,
121 |             ),
122 |         };
123 |     })(candidates, ctx.settings, principal);
124 |     return { selected: selection.selected, principal: principal, debug: selection.debug };
125 | }
126 | function shouldStopEarly(result, depth, stableDepthCount, rootStaticScore, settings) {
127 |     return (
128 |         !('hard' === settings.preset || depth < 4 || stableDepthCount < 1) &&
129 |         ((function rootScoreGap(debug) {
130 |             const [bestCandidate, secondCandidate] = debug.candidates;
131 |             return bestCandidate && secondCandidate
132 |             ? bestCandidate.score - secondCandidate.score
133 |             : 1e7;
134 |         })(result.debug) ≥ 120 ||
135 |         Math.abs(result.principal.score - rootStaticScore) ≤ 50 ||
136 |         stableDepthCount ≥ 2)
137 |     );
138 | }
139 | function negamax(chess, depth, alpha, beta, ply, ctx) {
140 |     if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
141 |     if (depth ≤ 0)
142 |         return ctx.settings.quiescence
143 |         ? quiescence(chess, alpha, beta, ply, ctx)
144 |         : evaluatePosition(chess);
145 |     let bestScore = -1e7,

```

```

146 |         currentAlpha = alpha;
147 |         const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
148 |         for (const move of legalMoves) {
149 |             chess.move(toMoveCommand(move));
150 |             const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
151 |             if (
152 |                 (chess.undo(),
153 |                 (bestScore = Math.max(bestScore, score)),
154 |                 (currentAlpha = Math.max(currentAlpha, score)),
155 |                 currentAlpha ≥ beta)
156 |             )
157 |                 break;
158 |         }
159 |         return bestScore;
160 |     }
161 |     function quiescence(chess, alpha, beta, ply, ctx) {
162 |         if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
163 |         let currentAlpha = alpha;
164 |         const standPat = evaluatePosition(chess);
165 |         if (standPat ≥ beta) return beta;
166 |         if (((currentAlpha = Math.max(currentAlpha, standPat)), ply ≥ 6)) return currentAlpha;
167 |         const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
168 |             tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
169 |         for (const move of tacticalMoves) {
170 |             chess.move(toMoveCommand(move));
171 |             const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
172 |             if ((chess.undo(), score ≥ beta)) return beta;
173 |             currentAlpha = Math.max(currentAlpha, score);
174 |         }
175 |         return currentAlpha;
176 |     }
177 |     function evaluatePosition(chess) {
178 |         const board = chess.board(),
179 |             pawns = [],
180 |             bishops = { w: 0, b: 0 },
181 |             kings = {};
182 |         let whiteScore = 0,
183 |             blackScore = 0,
184 |             nonPawnMaterial = 0;
185 |         for (const row of board)
186 |             for (const pieceOnSquare of row) {
187 |                 if (null === pieceOnSquare) continue;
188 |                 const { color: color, type: type, square: square } = pieceOnSquare,
189 |                     pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
190 |                 ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
191 |                 ('p' === type
192 |                 ? pawns.push({ color: color, file: fileIndex(square), rank: rankNumber(square) })
193 |                 : 'k' !== type && (nonPawnMaterial += PIECE_VALUES[type]),
194 |                 'b' === type && (bishops[color] += 1),
195 |                 'k' === type && (kings[color] = square));
196 |             }
197 |         ((whiteScore += bishops.w ≥ 2 ? 35 : 0),
198 |         (blackScore += bishops.b ≥ 2 ? 35 : 0),
199 |         (whiteScore += pawnStructureScore(pawns, 'w')),
200 |         (blackScore += pawnStructureScore(pawns, 'b')),
201 |         (whiteScore += openingPrinciplesScore(board, pawns, 'w', nonPawnMaterial)),
202 |         (blackScore += openingPrinciplesScore(board, pawns, 'b', nonPawnMaterial)),
203 |         (whiteScore += developmentScore(board, 'w')),
204 |         (blackScore += developmentScore(board, 'b')),
205 |         (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
206 |         (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial)));
207 |         let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
208 |         return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
209 |     }
210 |     function pieceSquareScore(type, color, square) {
211 |         const file = fileIndex(square),
212 |             relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
213 |             centerDistance = Math.abs(file - 3.5) + Math.abs(rankNumber(square) - 4.5);
214 |         switch (type) {
215 |             case 'p':
216 |                 return 7 * relativeRank + (file ≥ 2 && file ≤ 5 ? 8 : 0);

```

```

217 |         case 'n':
218 |             return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
219 |         case 'b':
220 |             return 30 - 7 * centerDistance;
221 |         case 'r':
222 |             return (relativeRank >= 7 ? 18 : 0) + (0 === file || 7 === file ? 4 : 0);
223 |         case 'q':
224 |             return 14 - 3 * centerDistance;
225 |         case 'k':
226 |             return relativeRank >= 7 ? 18 : 0;
227 |     }
228 | }
229 |
230 | function pawnStructureScore(pawns, color) {
231 |     const ownPawns = pawns.filter((pawn) => pawn.color === color),
232 |     fileCounts = Array.from({ length: 8 }, () => 0);
233 |     let score = 0;
234 |     for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
235 |     for (const pawn of ownPawns) {
236 |         const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
237 |         (fileCounts[pawn.file] > 1 && (score -= 14),
238 |         0 === (fileCounts[pawn.file - 1] ?? 0) &&
239 |         0 === (fileCounts[pawn.file + 1] ?? 0) &&
240 |         (score -= 10),
241 |         !isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
242 |     }
243 |     return score;
244 | }
245 |
246 | function isPassedPawn(pawn, pawns) {
247 |     const enemyColor = (function oppositeColor(color) {
248 |         return 'w' === color ? 'b' : 'w';
249 |     })(pawn.color);
250 |     return !pawns.some(
251 |         (candidate) =>
252 |         !(candidate.color !== enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
253 |         ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
254 |     );
255 | }
256 |
257 | function openingPhaseWeight(nonPawnMaterial) {
258 |     return clamp(
259 |         (nonPawnMaterial - OPENING_PHASE_END_MATERIAL) /
260 |         (STARTING_NON_PAWN_MATERIAL - OPENING_PHASE_END_MATERIAL),
261 |         0,
262 |         1,
263 |     );
264 | }
265 |
266 | function openingPrinciplesScore(board, pawns, color, nonPawnMaterial) {
267 |     const phase = openingPhaseWeight(nonPawnMaterial);
268 |     if (phase <= 0) return 0;
269 |     let score = (function centralLineOpeningScore(board, color) {
270 |         let score = 0;
271 |         return (
272 |             hasPawnOnSquare(board, color, 'w' === color ? 'd2' : 'd7') || (score += 10),
273 |             hasPawnOnSquare(board, color, 'w' === color ? 'e2' : 'e7') || (score += 12),
274 |             score
275 |         );
276 |     })(board, color),
277 |     developedCenterPawns = 0;
278 |     for (const pawn of pawns) {
279 |         if (pawn.color !== color) continue;
280 |         const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
281 |         3 === pawn.file || 4 === pawn.file
282 |         ? (4 === relativeRank || 5 === relativeRank
283 |         ? (score += 76)
284 |         : 3 === relativeRank && (score += 30),
285 |         relativeRank >= 3 && (developedCenterPawns += 1))
286 |         : (2 !== pawn.file && 5 !== pawn.file) ||
287 |         (4 === relativeRank || 5 === relativeRank
288 |         ? (score += 18)
289 |         : 3 === relativeRank && (score += 6));
290 |     }
291 |     return (developedCenterPawns >= 2 && (score += 16), Math.round(score * phase));

```

```

288 |     }
289 |     function hasPawnOnSquare(board, color, square) {
290 |         for (const row of board)
291 |             for (const piece of row)
292 |                 if (piece?.color === color && 'p' === piece.type && piece.square === square) return !0;
293 |         return !1;
294 |     }
295 |     function developmentScore(board, color) {
296 |         const homeRank = 'w' === color ? '1' : '8',
297 |               knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
298 |               bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
299 |         let score = 0;
300 |         for (const row of board)
301 |             for (const piece of row)
302 |                 null !== piece &&
303 |                 piece.color === color &&
304 |                 ('n' !== piece.type || knightHomeSquares.has(piece.square) || (score += 18),
305 |                 'b' !== piece.type || bishopHomeSquares.has(piece.square) || (score += 16),
306 |                 'q' === piece.type &&
307 |                 piece.square.endsWith(homeRank) &&
308 |                 hasMovedMinorPieces(board, color) &&
309 |                 (score -= 12));
310 |         return score;
311 |     }
312 |     function hasMovedMinorPieces(board, color) {
313 |         const homeSquares =
314 |             'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
315 |         for (const row of board)
316 |             for (const piece of row)
317 |                 if (
318 |                     null !== piece &&
319 |                     piece.color === color &&
320 |                     ('n' === piece.type || 'b' === piece.type) &&
321 |                     !homeSquares.has(piece.square)
322 |                 )
323 |                     return !1;
324 |         return !0;
325 |     }
326 |     function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
327 |         if (!kingSquare || nonPawnMaterial < 1800) return 0;
328 |         const kingFile = fileIndex(kingSquare),
329 |               kingRank = rankNumber(kingSquare),
330 |               shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
331 |         let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
332 |         for (let file = kingFile - 1; file ≤ kingFile + 1; file += 1)
333 |             file < 0 ||
334 |             file > 8 ||
335 |             (pawns.some(
336 |                 (pawn) => pawn.color === color && pawn.file === file && pawn.rank === shieldRank,
337 |             )
338 |             ? (score += 14)
339 |             : (score +=
340 |                 openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) ?? -10));
341 |         return score;
342 |     }
343 |     function openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) {
344 |         const homeKingSquare = 'w' === color ? 'e1' : 'e8',
345 |               phase = openingPhaseWeight(nonPawnMaterial);
346 |         return kingSquare !== homeKingSquare || phase ≤ 0 || (3 !== file && 4 !== file)
347 |             ? null
348 |             : pawns.some((pawn) => {
349 |                 if (pawn.color !== color || pawn.file !== file) return !1;
350 |                 const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
351 |                 return 3 === relativeRank || 4 === relativeRank;
352 |             })
353 |             ? Math.round(24 * phase - 10)
354 |             : null;
355 |     }
356 |     function terminalScore(chess, ply) {
357 |         return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
358 |     }

```



```

359 |     function orderMoves(moves, preferredMove) {
360 |         return [...moves].sort(
361 |             (left, right) =>
362 |                 moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove),
363 |             );
364 |     }
365 |     function moveOrderingScore(move, preferredMove) {
366 |         let score = 0;
367 |         return (
368 |             preferredMove &&
369 |             (function isSameMove(move, command) {
370 |                 return (
371 |                     move.from === command.from &&
372 |                     move.to === command.to &&
373 |                     (move.promotion ?? void 0) === (command.promotion ?? void 0)
374 |                 );
375 |             })(move, preferredMove) &&
376 |             (score += 2e6),
377 |             move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
378 |             move.isCapture() &&
379 |             (score +=
380 |                 12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) -
381 |                 PIECE_VALUES[move.piece]),
382 |             move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] + 8e3),
383 |             (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
384 |             (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
385 |             score
386 |         );
387 |     }
388 |     function checkTime(ctx) {
389 |         if (((ctx.nodes += 1), ctx.nodes % 2048 !== 0)) return;
390 |         const now = Date.now();
391 |         if (
392 |             (now - ctx.lastProgressAt >= 100 &&
393 |             publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now),
394 |             now >= ctx.deadline)
395 |         )
396 |             throw new SearchTimeout();
397 |     }
398 |     function publishSearchProgress(
399 |         ctx,
400 |         progressState,
401 |         elapsedMs = Date.now() - ctx.startTime,
402 |         progressAt = Date.now(),
403 |     ) {
404 |         ((ctx.progressState = progressState),
405 |         (ctx.lastProgressAt = progressAt),
406 |         ctx.onProgress?.({
407 |             kind: 'progress',
408 |             id: ctx.requestId,
409 |             move: progressState.move,
410 |             depthReached: progressState.depthReached,
411 |             nodes: ctx.nodes,
412 |             elapsedMs: elapsedMs,
413 |             score: progressState.score,
414 |             debug: progressState.debug,
415 |         })));
416 |     }
417 |     function isTacticalMove(move) {
418 |         return (
419 |             move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
420 |         );
421 |     }
422 |     function createDeterministicDebug(
423 |         candidates,
424 |         principal,
425 |         selected,
426 |         windowCp,
427 |         floorScore,
428 |         ticketPower,
429 |     ) {

```

```

430 |         return {
431 |             candidates: sortCandidatesByScore(candidates).map((candidate) => ({
432 |                 move: candidate.move,
433 |                 san: candidate.san,
434 |                 score: candidate.score,
435 |                 tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
436 |                 probability: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
437 |                 selected: isSameMoveCommand(candidate.move, selected.move),
438 |                 principal: isSameMoveCommand(candidate.move, principal.move),
439 |             })),
440 |             bestScore: principal.score,
441 |             windowCp: windowCp,
442 |             floorScore: floorScore,
443 |             ticketPower: ticketPower,
444 |             totalTickets: 1,
445 |         };
446 |     }
447 |     function createWeightedDebug(
448 |         candidates,
449 |         principal,
450 |         selected,
451 |         weightedCandidates,
452 |         windowCp,
453 |         floorScore,
454 |         ticketPower,
455 |         totalTickets,
456 |     ) {
457 |         return {
458 |             candidates: sortCandidatesByScore(candidates).map((candidate) => {
459 |                 const weightedCandidate = weightedCandidates.find(({ candidate: weighted }) =>
460 |                     isSameMoveCommand(weighted.move, candidate.move),
461 |                 ),
462 |                 tickets = weightedCandidate?.tickets ?? 0;
463 |                 return {
464 |                     move: candidate.move,
465 |                     san: candidate.san,
466 |                     score: candidate.score,
467 |                     tickets: tickets,
468 |                     probability: totalTickets > 0 ? tickets / totalTickets : 0,
469 |                     selected: isSameMoveCommand(candidate.move, selected.move),
470 |                     principal: isSameMoveCommand(candidate.move, principal.move),
471 |                 };
472 |             }),
473 |             bestScore: principal.score,
474 |             windowCp: windowCp,
475 |             floorScore: floorScore,
476 |             ticketPower: ticketPower,
477 |             totalTickets: totalTickets,
478 |         };
479 |     }
480 |     function sortCandidatesByScore(candidates) {
481 |         return [...candidates].sort((left, right) => right.score - left.score);
482 |     }
483 |     function toMoveCommand(move) {
484 |         return move.promotion
485 |             ? { from: move.from, to: move.to, promotion: move.promotion }
486 |             : { from: move.from, to: move.to };
487 |     }
488 |     function isSameMoveCommand(left, right) {
489 |         return (
490 |             left.from === right.from &&
491 |             left.to === right.to &&
492 |             (left.promotion ?? void 0) === (right.promotion ?? void 0)
493 |         );
494 |     }
495 |     function fileIndex(square) {
496 |         return square.charCodeAt(0) - 97;
497 |     }
498 |     function rankNumber(square) {
499 |         return Number(square[1]);
500 |     }

```

```

501 |     function clamp(value, min, max) {
502 |         return Math.min(max, Math.max(min, value));
503 |     }
504 |     class SearchTimeout extends Error {}
505 |     return (
506 |         (exports.AI_PRESETS = AI_PRESETS),
507 |         (exports.DEFAULT_AI_SETTINGS = DEFAULT_AI_SETTINGS),
508 |         (exports.normalizeAiSettings = normalizeAiSettings),
509 |         (exports.searchBestMove = function searchBestMove(request, onProgress) {
510 |             const chess = new chessRuntime_ts.Chess(request.fen),
511 |                 settings = normalizeAiSettings(request.settings),
512 |                 legalMoves = chess.moves({ verbose: !0 }),
513 |                 startTime = Date.now(),
514 |                 ctx = {
515 |                     requestId: request.id,
516 |                     settings: settings,
517 |                     deadline: startTime + settings.timeLimitMs,
518 |                     startTime: startTime,
519 |                     nodes: 0,
520 |                     onProgress: onProgress,
521 |                     lastProgressAt: startTime,
522 |                     progressState: { move: null, depthReached: 0, score: 0 },
523 |                 };
524 |             if (0 === legalMoves.length)
525 |                 return {
526 |                     id: request.id,
527 |                     move: null,
528 |                     depthReached: 0,
529 |                     nodes: 0,
530 |                     elapsedMs: 0,
531 |                     score: terminalScore(chess, 0),
532 |                 };
533 |             let debug,
534 |                 best = null,
535 |                 previousBestMove = null,
536 |                 previousPrincipal = null,
537 |                 depthReached = 0,
538 |                 stableDepthCount = 0;
539 |             const rootStaticScore = evaluatePosition(chess);
540 |             for (let depth = 1; depth ≤ settings.maxDepth; depth += 1) {
541 |                 try {
542 |                     const result = searchRoot(chess, depth, ctx, previousBestMove),
543 |                         elapsedMs = Date.now() - startTime;
544 |                     if (
545 |                         (best = result.selected),
546 |                         (debug = result.debug),
547 |                         (previousBestMove = result.principal.move),
548 |                         (depthReached = depth),
549 |                         publishSearchProgress(
550 |                             ctx,
551 |                             { move: best.move, depthReached: depthReached, score: best.score, debug: debug },
552 |                             elapsedMs,
553 |                         ),
554 |                         (stableDepthCount =
555 |                             null !== previousPrincipal &&
556 |                             isSameMoveCommand(previousPrincipal.move, result.principal.move) &&
557 |                             Math.abs(previousPrincipal.score - result.principal.score) ≤ 90
558 |                             ? stableDepthCount + 1
559 |                             : 0),
560 |                         shouldStopEarly(result, depth, stableDepthCount, rootStaticScore, settings))
561 |                         break;
562 |                     previousPrincipal = result.principal;
563 |                 } catch (error) {
564 |                     if (error instanceof SearchTimeout) break;
565 |                     throw error;
566 |                 }
567 |                 if (Date.now() ≥ ctx.deadline) break;
568 |             }
569 |             return (
570 |                 (best ??= (function chooseFallbackMove(chess) {
571 |

```

```

572 |         const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
573 |         if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
574 |         chess.move(toMoveCommand(move));
575 |         const score = evaluatePosition(chess);
576 |         return (chess.undo(), { move: toMoveCommand(move), score: score });
577 |     })(chess),
578 |     {
579 |         id: request.id,
580 |         move: best.move,
581 |         depthReached: depthReached,
582 |         nodes: ctx.nodes,
583 |         elapsedMs: Date.now() - startTime,
584 |         score: best.score,
585 |         debug: debug,
586 |     }
587 | );
588 | },
589 | Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' }),
590 | exports
591 | );
592 | }({}, globalThis.__TSCS.chessRuntime);
593 |
594 | return __TSCS.ai;
595 | }
596 | var __TSCS.ai = __TSCS.createAi();
597 | globalThis.__TSCS = globalThis.__TSCS || {};
598 | globalThis.__TSCS.factories = globalThis.__TSCS.factories || {};
599 | globalThis.__TSCS.factories.ai = __TSCS.createAi();
600 | globalThis.__TSCS.ai = __TSCS.ai;
601 |

```

==== src/chess/app.js (37627 B / 37cf92152687) ====

```

1 | var __TSCSApp = (function (exports, ai_ts, chessRuntime_ts) {
2 |     'use strict';
3 |     const jsContent = (() => {
4 |         const element = document.getElementById('ai-worker-source-1');
5 |         const registry = globalThis.__TSCS;
6 |         const chessCoreFactory = registry?.factories?.chessCore;
7 |         const aiFactory = registry?.factories?.ai;
8 |         if (!element) {
9 |             throw new Error('Missing inline worker bootstrap: ai-worker-source-1');
10 |         }
11 |         if (!chessCoreFactory || !aiFactory) {
12 |             throw new Error('Missing serializable chess worker factories');
13 |         }
14 |         return [
15 |             `globalThis.__TSCS = globalThis.__TSCS || {};`,
16 |             `globalThis.TSCSShared = (${chessCoreFactory.toString})();`,
17 |             `globalThis.__TSCS.chessRuntime = {`,
18 |             `  Chess: globalThis.TSCSShared.Chess,`,
19 |             `  SQUARES: globalThis.TSCSShared.SQUARES,`,
20 |             `};`,
21 |             `globalThis.__TSCS.ai = (${aiFactory.toString})();`,
22 |             `element.textContent += `,
23 |             `].join('\n');
24 |         })(),
25 |         blob =
26 |             'undefined' !== typeof self &&
27 |             self.Blob &&
28 |             new Blob(['(self.URL || self.webkitURL).revokeObjectURL(self.location.href);', jsContent], {
29 |                 type: 'text/javascript; charset=utf-8',
30 |             });
31 |         function WorkerWrapper(options) {
32 |             let objURL;
33 |             try {
34 |                 if (!((objURL = blob && (self.URL || self.webkitURL).createObjectURL(blob)), !objURL))
35 |                     throw '';
36 |                 const worker = new Worker(objURL, { name: options?.name });
37 |                 return (
38 |                     worker.addEventListener('error', () => {
39 |                         (self.URL || self.webkitURL).revokeObjectURL(objURL);

```

```

40 |         }},
41 |         worker
42 |     });
43 |     } catch (e) {
44 |         return new Worker('data:text/javascript;charset=utf-8,' + encodeURIComponent(jsContent), {
45 |             name: options?.name,
46 |         });
47 |     }
48 | }
49 | const BOARD_FILES = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
50 |     BLACK_FILES = [...BOARD_FILES].reverse(),
51 |     WHITE_RANKS = [8, 7, 6, 5, 4, 3, 2, 1],
52 |     BLACK_RANKS = [...WHITE_RANKS].reverse(),
53 |     PROMOTION_PIECES = ['q', 'r', 'b', 'n'],
54 |     SQUARE_SET = new Set(chessRuntime.ts.SQUARES),
55 |     AI_WHITE_OPENING_PRESET = [
56 |         { from: 'e2', to: 'e4', weight: 29 },
57 |         { from: 'd2', to: 'd4', weight: 29 },
58 |         { from: 'g1', to: 'f3', weight: 28 },
59 |         { from: 'c2', to: 'c4', weight: 22 },
60 |         { from: 'g2', to: 'g3', weight: 22 },
61 |         { from: 'e2', to: 'e3', weight: 12 },
62 |         { from: 'c2', to: 'c3', weight: 7 },
63 |     ],
64 |     CLOCK_PRESETS = [
65 |         { '1/1': { baseMs: 6e4, incrementMs: 1e3 },
66 |         { '3/2': { baseMs: 18e4, incrementMs: 2e3 },
67 |         { '5/3': { baseMs: 3e5, incrementMs: 3e3 },
68 |         { '10/5': { baseMs: 6e5, incrementMs: 5e3 },
69 |         { '15/10': { baseMs: 9e5, incrementMs: 1e4 },
70 |     ],
71 |     PIECE_GLYPHS = {
72 |         w: { p: '♔', n: '♖', b: '♜', r: '♞', q: '♝', k: '♚' },
73 |         b: { p: '♚', n: '♞', b: '♜', r: '♖', q: '♝', k: '♔' },
74 |     };
75 |     function requireElement(selector, root) {
76 |         const element = root.querySelector(selector);
77 |         if (!element) throw new Error(`Missing element: ${selector}`);
78 |         return element;
79 |     }
80 |     function createCoordinate(type, label) {
81 |         const coordinate = document.createElement('span');
82 |         return (
83 |             (coordinate.className = `coordinate coordinate-${type}`),
84 |             (coordinate.textContent = label),
85 |             coordinate
86 |         );
87 |     }
88 |     function toSquare(file, rank) {
89 |         return `${file}${rank}`;
90 |     }
91 |     function createClockState(preset) {
92 |         const baseMs = CLOCK_PRESETS[preset].baseMs;
93 |         return { w: baseMs, b: baseMs };
94 |     }
95 |     function colorName(color) {
96 |         return 'w' === color ? '백' : '흑';
97 |     }
98 |     function presetName(preset) {
99 |         switch (preset) {
100 |             case 'easy':
101 |                 return '쉬움';
102 |             case 'normal':
103 |                 return '보통';
104 |             case 'hard':
105 |                 return '어려움';
106 |         }
107 |     }
108 |     function formatSeconds(milliseconds) {
109 |         return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 === 0 ? 0 : 2)}초`;
110 |     }

```

```

111 |     function formatDebugScore(score) {
112 |         return Math.abs(score) >= 9e5
113 |         ? score > 0
114 |         ? '+mate'
115 |         : '-mate'
116 |         : score > 0
117 |         ? `+${Math.round(score)}`
118 |         : String(Math.round(score));
119 |     }
120 |     function oppositeColor(color) {
121 |         return 'w' === color ? 'b' : 'w';
122 |     }
123 |     function getSquareLabel(square, piece) {
124 |         return piece
125 |         ? `${square}-${colorName(piece.color)}-${pieceName(piece.type)}`
126 |         : `${square}빈칸`;
127 |     }
128 |     function pieceName(piece) {
129 |         switch (piece) {
130 |             case 'p':
131 |                 return '폰';
132 |             case 'n':
133 |                 return '나이트';
134 |             case 'b':
135 |                 return '비숍';
136 |             case 'r':
137 |                 return '룩';
138 |             case 'q':
139 |                 return '퀸';
140 |             case 'k':
141 |                 return '킹';
142 |         }
143 |     }
144 |     return (
145 |         (exports.createChessApp = function createChessApp() {
146 |             const root = requireElement('#chess-app', document),
147 |                 boardElement = requireElement('#chess-board', root),
148 |                 colorControlsElement = requireElement('#color-controls', root),
149 |                 aiControlsElement = requireElement('#ai-controls', root),
150 |                 statusElement = requireElement('#game-status', root),
151 |                 turnMetaElement = requireElement('#turn-meta', root),
152 |                 modeMetaElement = requireElement('#mode-meta', root),
153 |                 moveMetaElement = requireElement('#move-meta', root),
154 |                 aiCandidatePanelElement = requireElement('#ai-candidate-panel', root),
155 |                 aiCandidateMetaElement = requireElement('#ai-candidate-meta', root),
156 |                 aiInfoStatusElement = requireElement('#ai-info-status', root),
157 |                 aiInfoPresetElement = requireElement('#ai-info-preset', root),
158 |                 aiInfoHumanElement = requireElement('#ai-info-human', root),
159 |                 aiInfoDepthElement = requireElement('#ai-info-depth', root),
160 |                 aiInfoNodesElement = requireElement('#ai-info-nodes', root),
161 |                 aiInfoScoreElement = requireElement('#ai-info-score', root),
162 |                 aiCandidateListElement = requireElement('#ai-candidate-list', root),
163 |                 moveListElement = requireElement('#move-list', root),
164 |                 clockDisplayElement = requireElement('#clock-display', root),
165 |                 whiteClockElement = requireElement('#white-clock', root),
166 |                 blackClockElement = requireElement('#black-clock', root),
167 |                 whiteClockTimeElement = requireElement('#white-clock-time', root),
168 |                 blackClockTimeElement = requireElement('#black-clock-time', root),
169 |                 newGameButton = requireElement('#new-game-button', root),
170 |                 undoButton = requireElement('#undo-button', root),
171 |                 aiDepthInput = requireElement('#ai-depth-input', root),
172 |                 aiDepthValue = requireElement('#ai-depth-value', root),
173 |                 aiTimeInput = requireElement('#ai-time-input', root),
174 |                 aiTimeValue = requireElement('#ai-time-value', root),
175 |                 promotionOverlay = requireElement('#promotion-overlay', root),
176 |                 promotionChoices = requireElement('#promotion-choices', root),
177 |                 modeButtons = Array.from(root.querySelectorAll('[data-mode]')),
178 |                 colorButtons = Array.from(root.querySelectorAll('[data-color]')),
179 |                 presetButtons = Array.from(root.querySelectorAll('[data-preset]')),
180 |                 clockPresetButtons = Array.from(root.querySelectorAll('[data-clock-preset]')),
181 |                 game = new chessRuntime_ts.Chess();

```

```

182 |         let mode = 'local',
183 |         clockPreset = '10/5',
184 |         clockMs = createClockState(clockPreset),
185 |         clockHistory = [],
186 |         activeClockColor = null,
187 |         lastClockTick = Date.now(),
188 |         clockTimerId = null,
189 |         timedOutColor = null,
190 |         humanColor = 'w',
191 |         selectedSquare = null,
192 |         legalMoves = [],
193 |         pendingPromotion = null,
194 |         aiThinking = !1,
195 |         aiTimerId = null,
196 |         aiWorker = null,
197 |         aiRequestId = 0,
198 |         activeAiRequestId = null,
199 |         activeAiFen = null,
200 |         aiSettings = { ... ai_ts.DEFAULT_AI_SETTINGS },
201 |         aiProgress = null;
202 |         function selectSquare(square) {
203 |             ((selectedSquare = square),
204 |             (legalMoves = game.moves({ verbose: !0, square: square })),
205 |             render());
206 |         }
207 |         function playMove(moveCommand) {
208 |             if (!commitMove(moveCommand)) return (clearSelection(), void render());
209 |             ((pendingPromotion = null), clearSelection(), render(), scheduleAiIfNeeded());
210 |         }
211 |         function resetGame() {
212 |             (cancelAiMove(),
213 |             game.reset(),
214 |             (pendingPromotion = null),
215 |             clearSelection(),
216 |             resetClockState(),
217 |             restartClockForCurrentTurn(),
218 |             render(),
219 |             scheduleAiIfNeeded());
220 |         }
221 |         function scheduleAiIfNeeded() {
222 |             'ai' !== mode ||
223 |             game.isGameOver() ||
224 |             null !== timedOutColor ||
225 |             game.turn() === humanColor ||
226 |             aiThinking ||
227 |             pendingPromotion ||
228 |             (function tryCommitAiWhiteOpeningPreset() {
229 |                 if ('b' !== humanColor || 'w' !== game.turn() || game.history().length > 0) return !1;
230 |                 const selectedMove = (function pickWeightedOpeningMove(moves) {
231 |                     const totalWeight = moves.reduce((total, move) => total + move.weight, 0);
232 |                     if (totalWeight <= 0) return null;
233 |                     let draw = Math.random() * totalWeight;
234 |                     for (const move of moves)
235 |                         if (((draw -= move.weight), draw <= 0))
236 |                             return { from: move.from, to: move.to, promotion: move.promotion };
237 |                     const fallback = moves.at(-1);
238 |                     return fallback
239 |                     ? { from: fallback.from, to: fallback.to, promotion: fallback.promotion }
240 |                     : null;
241 |                 })()
242 |                 AI_WHITE_OPENING_PRESET.filter((presetMove) =>
243 |                 game.moves({ verbose: !0, square: presetMove.from }).some((move) =>
244 |                 (function isSameMoveCommand(move, command) {
245 |                     return (
246 |                         move.from === command.from &&
247 |                         move.to === command.to &&
248 |                         (move.promotion ?? void 0) === (command.promotion ?? void 0)
249 |                     );
250 |                 }))(move, presetMove),
251 |                 ),
252 |                 ),

```

```

253 |         );
254 |         return !(
255 |             !selectedMove ||
256 |             !commitMove(selectedMove) ||
257 |             ((aiProgress = null), clearSelection(), render(), 0)
258 |         );
259 |     })() ||
260 |     ((aiThinking = !0),
261 |     (aiProgress = null),
262 |     render(),
263 |     (aiTimerId = window.setTimeout(() => {
264 |         ((aiTimerId = null),
265 |         (function startAiSearch() {
266 |             aiWorker?.terminate();
267 |             const requestId = aiRequestId + 1,
268 |             fen = game.fen();
269 |             ((aiRequestId = requestId),
270 |             (activeAiRequestId = requestId),
271 |             (activeAiFen = fen),
272 |             (aiWorker = new WorkerWrapper()),
273 |             aiWorker.addEventListener('message', (event) => {
274 |                 !(function handleAiMessage(message) {
275 |                     if (
276 |                         message.id === activeAiRequestId &&
277 |                         null !== activeAiFen &&
278 |                         game.fen() === activeAiFen
279 |                     ) {
280 |                         if (((aiProgress = message), 'progress' === message.kind))
281 |                             return (renderStatus(), void renderAiCandidateDebug());
282 |                         ((aiThinking = !1),
283 |                         (activeAiRequestId = null),
284 |                         (activeAiFen = null),
285 |                         aiWorker?.terminate(),
286 |                         (aiWorker = null),
287 |                         message.move && commitMove(message.move),
288 |                         clearSelection(),
289 |                         render());
290 |                     }
291 |                 })(event.data);
292 |             })),
293 |             aiWorker.addEventListener('error', () => {
294 |                 activeAiRequestId === requestId &&
295 |                 ((aiThinking = !1),
296 |                 (activeAiRequestId = null),
297 |                 (activeAiFen = null),
298 |                 aiWorker?.terminate(),
299 |                 (aiWorker = null),
300 |                 render());
301 |             })),
302 |             aiWorker.postMessage({
303 |                 id: requestId,
304 |                 fen: fen,
305 |                 settings: resolveAiSettingsForSearch(),
306 |             }));
307 |         })());
308 |     }, 180));
309 | }
310 | function cancelAiMove() {
311 |     ((aiRequestId += 1),
312 |     (activeAiRequestId = null),
313 |     (activeAiFen = null),
314 |     (aiProgress = null),
315 |     null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
316 |     aiWorker && (aiWorker.terminate(), (aiWorker = null)),
317 |     (aiThinking = !1));
318 | }
319 | function commitMove(moveCommand) {
320 |     const movingColor = game.turn(),
321 |     clockSnapshot = { ... clockMs };
322 |     if (!flushClockElapsed()) return !1;
323 |     try {

```



```

324 |         game.move(moveCommand);
325 |     } catch {
326 |         return !1;
327 |     }
328 |     return (
329 |         clockHistory.push(clockSnapshot),
330 |         (clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
331 |         restartClockForCurrentTurn(),
332 |         !0
333 |     );
334 | }
335 | function updateAiSettings(settings) {
336 |     ((aiSettings = ai_ts.normalizeAiSettings(settings)),
337 |     aiThinking && cancelAiMove(),
338 |     render(),
339 |     scheduleAiIfNeeded());
340 | }
341 | function resolveAiSettingsForSearch() {
342 |     const settings = ai_ts.normalizeAiSettings(aiSettings);
343 |     if ('hard' !== settings.preset) return settings;
344 |     const aiColor = game.turn(),
345 |         remainingMs = clockMs[aiColor],
346 |         incrementMs = CLOCK_PRESETS[clockPreset].incrementMs,
347 |         targetMs =
348 |             (remainingMs / 30) *
349 |             (function openingTimeFactor(ply, hasCastled) {
350 |                 return hasCastled ? 1 : 0.2 ** (1 - Math.min(ply / 20, 1));
351 |             })(
352 |                 game.history().length,
353 |                 game
354 |                     .history({ verbose: !0 })
355 |                     .some(
356 |                         (move) =>
357 |                             move.color === aiColor &&
358 |                             (move.isKingsideCastle() || move.isQueensideCastle()),
359 |                         ),
360 |                     ) +
361 |             0.8 * incrementMs,
362 |         timeLimitMs = remainingMs ≤ 3e3 ? Math.min(targetMs, 0.35 * remainingMs) : targetMs;
363 |     return ai_ts.normalizeAiSettings({ ... settings, timeLimitMs: timeLimitMs });
364 | }
365 | function resetClockState() {
366 |     ((clockMs = createClockState(clockPreset)),
367 |     (clockHistory = []),
368 |     (timedOutColor = null),
369 |     (activeClockColor = null),
370 |     (lastClockTick = Date.now()));
371 | }
372 | function restartClockForCurrentTurn() {
373 |     if ((clearClockTimer(), game.isGameOver() || null !== timedOutColor))
374 |         return ((activeClockColor = null), void renderClocks());
375 |     ((activeClockColor = game.turn()),
376 |     (lastClockTick = Date.now()),
377 |     (clockTimerId = window.setInterval(tickClock, 100)),
378 |     renderClocks());
379 | }
380 | function clearClockTimer() {
381 |     null !== clockTimerId && (window.clearInterval(clockTimerId), (clockTimerId = null));
382 | }
383 | function tickClock() {
384 |     flushClockElapsed() && renderClocks();
385 | }
386 | function flushClockElapsed() {
387 |     if (null === activeClockColor || null !== timedOutColor || game.isGameOver()) return !0;
388 |     const now = Date.now(),
389 |         elapsedMs = Math.max(0, now - lastClockTick);
390 |     return (
391 |         (lastClockTick = now),
392 |         0 === elapsedMs ||
393 |         ((clockMs[activeClockColor] = Math.max(0, clockMs[activeClockColor] - elapsedMs)),
394 |         0 !== clockMs[activeClockColor] ||

```

```

395 |         ((function handleClockTimeout(color) {
396 |             ((timedOutColor = color),
397 |             (activeClockColor = null),
398 |             (clockMs[color] = 0),
399 |             clearClockTimer(),
400 |             cancelAiMove(),
401 |             clearSelection(),
402 |             render());
403 |         })(activeClockColor),
404 |         !1))
405 |     );
406 | }
407 |
408 | function clearSelection() {
409 |     ((selectedSquare = null), (legalMoves = []));
410 | }
411 |
412 | function render() {
413 |     (!function renderControls() {
414 |         (modeButtons.forEach((button) => {
415 |             const isActive = button.dataset.mode === mode;
416 |             (button.classList.toggle('is-active', isActive),
417 |             button.setAttribute('aria-pressed', String(isActive)));
418 |         })),
419 |         colorButtons.forEach((button) => {
420 |             const isActive = button.dataset.color === humanColor;
421 |             (button.classList.toggle('is-active', isActive),
422 |             button.setAttribute('aria-pressed', String(isActive)));
423 |         })),
424 |         (colorControlsElement.hidden = 'ai' !== mode),
425 |         (aiControlsElement.hidden = 'ai' !== mode),
426 |         (function renderAiSettingsControls() {
427 |             (presetButtons.forEach((button) => {
428 |                 const isActive = button.dataset.preset === aiSettings.preset;
429 |                 (button.classList.toggle('is-active', isActive),
430 |                 button.setAttribute('aria-pressed', String(isActive)));
431 |             })),
432 |             (aiDepthInput.value =
433 |             'hard' === aiSettings.preset ? aiDepthInput.max : String(aiSettings.maxDepth)),
434 |             (aiDepthInput.disabled = 'hard' === aiSettings.preset),
435 |             (aiDepthValue.textContent =
436 |             'hard' === aiSettings.preset ? '무제한' : `${aiSettings.maxDepth}`),
437 |             (aiTimeInput.value = String(aiSettings.timeLimitMs)),
438 |             (aiTimeInput.disabled = 'hard' === aiSettings.preset),
439 |             (aiTimeValue.textContent =
440 |             'hard' === aiSettings.preset
441 |             ? '자동'
442 |             : `${formatSeconds(aiSettings.timeLimitMs)}`));
443 |         })(),
444 |         (function renderClockSettingsControls() {
445 |             clockPresetButtons.forEach((button) => {
446 |                 const isActive = button.dataset.clockPreset === clockPreset;
447 |                 (button.classList.toggle('is-active', isActive),
448 |                 button.setAttribute('aria-pressed', String(isActive)));
449 |             }));
450 |         })(),
451 |         (undoButton.disabled = 0 === game.history().length));
452 |     })(),
453 |     renderClocks(),
454 |     (function renderBoard() {
455 |         const fragment = document.createDocumentFragment(),
456 |             orientation = getBoardOrientation(),
457 |             files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
458 |             ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
459 |             lastMove = (function getLastMove() {
460 |                 return game.history({ verbose: !0 }).at(-1) ?? null;
461 |             })();
462 |         boardElement.classList.toggle('is-locked', !canUseBoard());
463 |         for (const rank of ranks)
464 |             for (const file of files) {
465 |                 const square = toSquare(file, rank),
466 |                     piece = game.get(square),
467 |                     button = document.createElement('button');

```

```

466 |         if (
467 |             ((button.type === 'button'),
468 |             (button.className === getSquareClasses(square, lastMove)),
469 |             (button.dataset.square === square),
470 |             (button.disabled === !canUseBoard()),
471 |             button.setAttribute('role', 'gridcell'),
472 |             button.setAttribute('aria-label', getSquareLabel(square, piece)),
473 |             file === files[0] && button.appendChild(createCoordinate('rank', String(rank))),
474 |             rank === ranks[ranks.length - 1] &&
475 |             button.appendChild(createCoordinate('file', file)),
476 |             piece)
477 |         ) {
478 |             const pieceElement = document.createElement('span');
479 |             ((pieceElement.className === 'piece'),
480 |             (pieceElement.textContent === PIECE_GLYPHS[piece.color][piece.type]),
481 |             button.appendChild(pieceElement));
482 |         }
483 |         fragment.appendChild(button);
484 |     }
485 |     boardElement.replaceChildren(fragment);
486 | }(),
487 | renderStatus(),
488 | renderAiCandidateDebug(),
489 | (function renderMoveList() {
490 |     const history = game.history({ verbose: !0 });
491 |     fragment = document.createDocumentFragment();
492 |     if (0 === history.length) {
493 |         const empty = document.createElement('p');
494 |         return (
495 |             (empty.className === 'move-empty'),
496 |             (empty.textContent === '아직 둔 수가 없습니다.'),
497 |             fragment.appendChild(empty),
498 |             void moveListElement.replaceChildren(fragment)
499 |         );
500 |     }
501 |     for (let index = 0; index < history.length; index += 2) {
502 |         const row = document.createElement('div'),
503 |             moveNumber = document.createElement('span'),
504 |             whiteMove = document.createElement('span'),
505 |             blackMove = document.createElement('span');
506 |         ((row.className === 'move-row'),
507 |         (moveNumber.className === 'move-number'),
508 |         (whiteMove.className === 'move-san'),
509 |         (blackMove.className === 'move-san'),
510 |         (moveNumber.textContent === `${Math.floor(index / 2) + 1}.`),
511 |         (whiteMove.textContent === history[index] ? '.san' : ''),
512 |         (blackMove.textContent === history[index + 1] ? '.san' : ''),
513 |         row.append(moveNumber, whiteMove, blackMove),
514 |         fragment.appendChild(row));
515 |     }
516 |     (moveListElement.replaceChildren(fragment),
517 |     (moveListElement.scrollTop = moveListElement.scrollHeight));
518 | }(),
519 | (function renderPromotionDialog() {
520 |     if (
521 |         ((promotionOverlay.hidden === null === pendingPromotion),
522 |         promotionChoices.replaceChildren(),
523 |         pendingPromotion)
524 |     )
525 |         for (const piece of PROMOTION_PIECES) {
526 |             const button = document.createElement('button');
527 |             ((button.className === 'promotion-button'),
528 |             (button.type === 'button'),
529 |             (button.dataset.promotion === piece),
530 |             button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
531 |             (button.textContent === PIECE_GLYPHS[game.turn()][piece]),
532 |             promotionChoices.appendChild(button));
533 |         }
534 |     }(),
535 | }
536 | function renderClocks() {

```

```

537 |         const bottomColor = getBoardOrientation(),
538 |         topClockElement =
539 |         'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
540 |         bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
541 |         ((clockDisplayElement.children[0] === topClockElement &&
542 |         clockDisplayElement.children[1] === bottomClockElement) ||
543 |         clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
544 |         renderClockPosition('w', bottomColor),
545 |         renderClockPosition('b', bottomColor),
546 |         renderClock('w', whiteClockElement, whiteClockTimeElement),
547 |         renderClock('b', blackClockElement, blackClockTimeElement));
548 |     }
549 |     function renderClockPosition(color, bottomColor) {
550 |         const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
551 |             isBottom = color === bottomColor;
552 |         clockElement.classList.toggle('is-bottom', isBottom),
553 |         clockElement.classList.toggle('is-top', !isBottom);
554 |     }
555 |     function renderClock(color, clockElement, timeElement) {
556 |         const remainingMs = clockMs[color],
557 |             isActive = activeClockColor === color && null !== timedOutColor,
558 |             isTimedOut = timedOutColor === color;
559 |         (clockElement.classList.toggle('is-active', isActive),
560 |         clockElement.classList.toggle('is-warning', remainingMs ≤ 3e4),
561 |         clockElement.classList.toggle('is-danger', remainingMs ≤ 1e4),
562 |         clockElement.classList.toggle('is-timeout', isTimedOut),
563 |         (timeElement.textContent = (function formatClockTime(milliseconds) {
564 |             const safeMilliseconds = Math.max(0, milliseconds);
565 |             if (safeMilliseconds < 1e4)
566 |                 return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
567 |             const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
568 |             return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
569 |         }))(remainingMs));
570 |     }
571 |     function renderStatus() {
572 |         const historyLength = game.history().length;
573 |         ((statusElement.textContent = (function getStatusText() {
574 |             return game.isCheckmate()
575 |                 ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
576 |                 : game.isStalemate()
577 |                 ? '스테일메이트 무승부'
578 |                 : game.isInsufficientMaterial()
579 |                 ? '기물 부족 무승부'
580 |                 : game.isThreefoldRepetition()
581 |                 ? '3회 반복 무승부'
582 |                 : game.isDrawByFiftyMoves()
583 |                 ? '50수 규칙 무승부'
584 |                 : game.isDraw()
585 |                 ? '무승부'
586 |                 : timedOutColor
587 |                 ? [
588 |                     `${colorName(timedOutColor)} 시간 초과`,
589 |                     `${colorName(oppositeColor(timedOutColor))} 승리`,
590 |                 ].join('...')
591 |                 : aiThinking
592 |                 ? 'AI 계산 중'
593 |                 : game.isCheck()
594 |                 ? `${colorName(game.turn())} 체크`
595 |                 : `${colorName(game.turn())} 차례`;
596 |         }()))),
597 |         (turnMetaElement.textContent = colorName(game.turn())),
598 |         (modeMetaElement.textContent =
599 |         'local' === mode
600 |         ? '로컬 2인'
601 |         : ['AI 상대', `인간-${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
602 |         '...'),
603 |         (moveMetaElement.textContent = [
604 |             `${game.moveNumber()}수`,
605 |             `${historyLength} half-move`,
606 |         ].join('...')));

```

```

608 |     }
609 |     function renderAiCandidateDebug() {
610 |         if ('ai' !== mode)
611 |             return (
612 |                 (aiCandidatePanelElement.hidden !== !0),
613 |                 (aiCandidateMetaElement.textContent !== ''),
614 |                 (aiInfoStatusElement.textContent !== ''),
615 |                 (aiInfoPresetElement.textContent !== ''),
616 |                 (aiInfoHumanElement.textContent !== ''),
617 |                 (aiInfoDepthElement.textContent !== ''),
618 |                 (aiInfoNodesElement.textContent !== ''),
619 |                 (aiInfoScoreElement.textContent !== ''),
620 |                 void aiCandidateListElement.replaceChildren()
621 |             );
622 |         const debug = aiProgress?.debug;
623 |         if (
624 |             ((aiCandidatePanelElement.hidden !== !1),
625 |             (aiCandidateMetaElement.textContent = aiProgress
626 |             ? `경과 ${formatSeconds(aiProgress.elapsedMs)}`
627 |             : '')),
628 |             (aiInfoStatusElement.textContent = (function getAiInfoStatusText() {
629 |                 return game.isGameOver() || null !== timedOutColor
630 |                 ? '대국 종료'
631 |                 : aiThinking
632 |                 ? aiProgress
633 |                 ? '계산 중'
634 |                 : '계산 준비 중'
635 |                 : game.turn() === humanColor
636 |                 ? '대기 중'
637 |                 : '응수 대기';
638 |             })()),
639 |             (aiInfoPresetElement.textContent = presetName(aiSettings.preset)),
640 |             (aiInfoHumanElement.textContent = colorName(humanColor)),
641 |             (aiInfoDepthElement.textContent = aiProgress ? String(aiProgress.depthReached) : '-'),
642 |             (aiInfoNodesElement.textContent = aiProgress
643 |             ? `${(function formatNodes(nodes) {
644 |                 return nodes ≥ 1e6 ? `${Math.round(nodes / 1e3)}k` : String(nodes);
645 |             })(aiProgress.nodes)} nodes`
646 |             : '-'),
647 |             (aiInfoScoreElement.textContent = aiProgress ? formatDebugScore(aiProgress.score) : '-'),
648 |             !debug || 0 === debug.candidates.length)
649 |         )
650 |         return void aiCandidateListElement.replaceChildren();
651 |         aiCandidateMetaElement.textContent = [
652 |             aiCandidateMetaElement.textContent,
653 |             `best ${formatDebugScore(debug.bestScore)}`,
654 |             `창 ${Math.round(debug.windowCp)}`,
655 |             `p ${debug.ticketPower.toFixed(2)}`,
656 |         ].join(' ');
657 |         const fragment = document.createDocumentFragment();
658 |         for (const candidate of debug.candidates)
659 |             fragment.appendChild(createAiCandidateRow(candidate));
660 |         aiCandidateListElement.replaceChildren(fragment);
661 |     }
662 |     function createAiCandidateRow(candidate) {
663 |         const row = document.createElement('div');
664 |         ((row.className = 'ai-candidate-row'),
665 |         row.classList.toggle('is-selected', candidate.selected),
666 |         row.classList.toggle('is-principal', candidate.principal),
667 |         row.setAttribute('role', 'row'));
668 |         const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
669 |         return (
670 |             candidate.selected & sanCell.appendChild(createAiCandidateBadge('선택')),
671 |             candidate.principal & sanCell.appendChild(createAiCandidateBadge('최선')),
672 |             row.append(
673 |                 sanCell,
674 |                 createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
675 |                 createAiCandidateCell(
676 |                     'ai-candidate-weight',
677 |                     (function formatDebugWeight(weight) {
678 |                         return 0 === weight

```

```

679 |         ? '0'
680 |         : weight < 0.001
681 |         ? '<0.001'
682 |         : weight.toFixed(3).replace(/0+$/, '').replace(/\.$/, '');
683 |     })(candidate.tickets),
684 |     ),
685 |     createAiCandidateCell(
686 |         'ai-candidate-probability',
687 |         (function formatDebugProbability(probability) {
688 |             const percent = 100 * probability;
689 |             return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : `${percent.toFixed(1)}%`;
690 |         })(candidate.probability),
691 |     ),
692 |     ),
693 |     row
694 | );
695 | }
696 | function createAiCandidateCell(className, text) {
697 |     const cell = document.createElement('span');
698 |     return (
699 |         (cell.className = className),
700 |         cell.setAttribute('role', 'cell'),
701 |         (cell.textContent = text),
702 |         cell
703 |     );
704 | }
705 | function createAiCandidateBadge(text) {
706 |     const badge = document.createElement('span');
707 |     return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);
708 | }
709 | function getSquareClasses(square, lastMove) {
710 |     const classes = [
711 |         'board-square',
712 |         'light' === game.squareColor(square) ? 'is-light' : 'is-dark',
713 |     ],
714 |     destinationMove = legalMoves.find((move) => move.to === square);
715 |     return (
716 |         selectedSquare === square && classes.push('is-selected'),
717 |         destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
718 |         !lastMove ||
719 |         (lastMove.from !== square && lastMove.to !== square) ||
720 |         classes.push('is-last-move'),
721 |         (function isCheckedKing(square) {
722 |             const piece = game.get(square);
723 |             return Boolean(
724 |                 piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
725 |             );
726 |         })(square) && classes.push('is-check'),
727 |         classes.join(' ')
728 |     );
729 | }
730 | function getBoardOrientation() {
731 |     return 'ai' === mode ? humanColor : game.turn();
732 | }
733 | function canUseBoard() {
734 |     return !(
735 |         game.isGameOver() ||
736 |         null !== timedOutColor ||
737 |         aiThinking ||
738 |         pendingPromotion ||
739 |         ('local' !== mode && game.turn() !== humanColor)
740 |     );
741 | }
742 | (boardElement.addEventListener('click', (event) => {
743 |     const button = (function getSquareButton(target, boardElement) {
744 |         if (!(target instanceof Element)) return null;
745 |         const button = target.closest('.board-square');
746 |         return button && boardElement.contains(button) ? button : null;
747 |     })(event.target, boardElement);
748 |     button &&
749 |     (function isSquare(value) {

```

```

750 |         return 'string' === typeof value && SQUARE_SET.has(value);
751 |     })(button.dataset.square) &&
752 |     (function handleSquareClick(square) {
753 |         if (!canUseBoard()) return;
754 |         const piece = game.get(square);
755 |         if (selectedSquare) {
756 |             const movesToDestination = legalMoves.filter((move) => move.to === square);
757 |             return movesToDestination.length > 0
758 |                 ? movesToDestination.some((move) => move.isPromotion())
759 |                 : ((pendingPromotion = {
760 |                     from: selectedSquare,
761 |                     to: square,
762 |                     moves: movesToDestination,
763 |                 })),
764 |             void render()
765 |             : void playMove({ from: selectedSquare, to: square })
766 |             : piece?.color === game.turn()
767 |             ? void selectSquare(square)
768 |             : (clearSelection(), void render());
769 |         }
770 |         piece?.color === game.turn() && selectSquare(square);
771 |     })(button.dataset.square);
772 |     },
773 |     modeButtons.forEach((button) => {
774 |         button.addEventListener('click', () => {
775 |             const nextMode = button.dataset.mode;
776 |             (function isGameMode(value) {
777 |                 return 'local' === value || 'ai' === value;
778 |             })(nextMode) &&
779 |             nextMode !== mode &&
780 |             ((mode = nextMode), resetGame());
781 |         });
782 |     }),
783 |     colorButtons.forEach((button) => {
784 |         button.addEventListener('click', () => {
785 |             const nextColor = button.dataset.color;
786 |             (function isColor(value) {
787 |                 return 'w' === value || 'b' === value;
788 |             })(nextColor) &&
789 |             nextColor !== humanColor &&
790 |             ((humanColor = nextColor), resetGame());
791 |         });
792 |     }),
793 |     presetButtons.forEach((button) => {
794 |         button.addEventListener('click', () => {
795 |             const preset = button.dataset.preset;
796 |             (function isAiPreset(value) {
797 |                 return 'easy' === value || 'normal' === value || 'hard' === value;
798 |             })(preset) && updateAiSettings({ ... ai_ts.AI_PRESETS[preset] });
799 |         });
800 |     }),
801 |     clockPresetButtons.forEach((button) => {
802 |         button.addEventListener('click', () => {
803 |             const preset = button.dataset.clockPreset;
804 |             (function isClockPreset(value) {
805 |                 return (
806 |                     '1/1' === value ||
807 |                     '3/2' === value ||
808 |                     '5/3' === value ||
809 |                     '10/5' === value ||
810 |                     '15/10' === value
811 |                 );
812 |             })(preset) &&
813 |             preset !== clockPreset &&
814 |             ((clockPreset = preset), resetGame());
815 |         });
816 |     }),
817 |     aiDepthInput.addEventListener('input', () => {
818 |         var value;
819 |         updateAiSettings({
820 |             ... aiSettings,

```

```

821 |         maxDepth:
822 |         | ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value))));
823 |         });
824 |     },
825 |     aiTimeInput.addEventListener('input', () => {
826 |         updateAiSettings({ ... aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
827 |     }),
828 |     newGameButton.addEventListener('click', resetGame),
829 |     undoButton.addEventListener('click', function undoLastTurn() {
830 |         if ((cancelAiMove(), (pendingPromotion = null), 0 === game.history().length))
831 |             return void render();
832 |         const desiredUndoCount = 'ai' === mode && game.turn() === humanColor ? 2 : 1,
833 |             undoCount = Math.min(desiredUndoCount, game.history().length);
834 |         let restoredClockMs;
835 |         for (let i = 0; i < undoCount; i += 1)
836 |             (game.undo(), (restoredClockMs = clockHistory.pop() ?? restoredClockMs));
837 |         (restoredClockMs && (clockMs = restoredClockMs),
838 |             (timedOutColor = null),
839 |             clearSelection(),
840 |             restartClockForCurrentTurn(),
841 |             render());
842 |     }),
843 |     promotionChoices.addEventListener('click', (event) => {
844 |         const target = event.target;
845 |         if (!(target instanceof Element)) return;
846 |         const button = target.closest('[data-promotion]'),
847 |             promotion = button?.dataset.promotion;
848 |         (function isPromotionPiece(value) {
849 |             return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
850 |         })(promotion) &&
851 |         (function finishPromotion(promotion) {
852 |             if (!pendingPromotion) return;
853 |             if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
854 |             const { from: from, to: to } = pendingPromotion;
855 |             ((pendingPromotion = null), playMove({ from: from, to: to, promotion: promotion }));
856 |         })(promotion);
857 |     }),
858 |     resetClockState(),
859 |     restartClockForCurrentTurn(),
860 |     render());
861 | },
862 | Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' }),
863 | exports
864 | );
865 | }({}, globalThis.__TSCS.chess.ai, globalThis.__TSCS.chessRuntime);
866 |
867 | globalThis.__TSCS = globalThis.__TSCS || {};
868 | globalThis.__TSCS.app = __TSCSApp;
869 |

```

```

1 | function __TSCSChessCreateChessCore() {
2 |     var TSCSChessShared = (function (exports) {
3 |         'use strict';
4 |         const MASK64 = 0xfffffffffffffn;
5 |         function rotl(x, k) {
6 |             return 0xfffffffffffffn & ((x << k) | (x >> (64n - k)));
7 |         }
8 |         function wrappingMul(x, y) {
9 |             return (x * y) & MASK64;
10 |        }
11 |        function xoroshiro128(state) {
12 |            return function () {
13 |                let s0 = BigInt(state & MASK64),
14 |                    s1 = BigInt((state >> 64n) & MASK64);
15 |                const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
16 |                return (
17 |                    (s1 ^= s0),
18 |                    (s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64),
19 |                    (s1 = rotl(s1, 37n)),
20 |                    (state = (s1 << 64n) | s0),

```



```

21 |         result
22 |     );
23 | };
24 | }
25 | const rand = xoroshiro128(0xa187eb39cdcaed8f31c4b365b102e01en),
26 |     PIECE_KEYS = Array.from({ length: 2 }, () =>
27 |     Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand()))),
28 |     ,
29 |     EP_KEYS = Array.from({ length: 8 }, () => rand()),
30 |     CASTLING_KEYS = Array.from({ length: 16 }, () => rand()),
31 |     SIDE_KEY = rand(),
32 |     DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/PPPPPPPP/RNBQKBNR-w·KQkq-·-0·1';
33 | class Move {
34 |     color;
35 |     from;
36 |     to;
37 |     piece;
38 |     captured;
39 |     promotion;
40 |     flags;
41 |     san;
42 |     lan;
43 |     before;
44 |     after;
45 |     constructor(chess, internal) {
46 |         const {
47 |             color: color,
48 |             piece: piece,
49 |             from: from,
50 |             to: to,
51 |             flags: flags,
52 |             captured: captured,
53 |             promotion: promotion,
54 |         } = internal,
55 |             fromAlgebraic = algebraic(from),
56 |             toAlgebraic = algebraic(to);
57 |         ((this.color = color),
58 |         (this.piece = piece),
59 |         (this.from = fromAlgebraic),
60 |         (this.to = toAlgebraic),
61 |         (this.san = chess._moveToSan(internal, chess._moves({ legal: !0 }))),
62 |         (this.lan = fromAlgebraic + toAlgebraic),
63 |         (this.before = chess.fen()),
64 |         chess._makeMove(internal),
65 |         (this.after = chess.fen()),
66 |         chess._undoMove(),
67 |         (this.flags = ''));
68 |         for (const flag in BITS) BITS[flag] & flags && (this.flags += FLAGS[flag]);
69 |         (captured && (this.captured = captured),
70 |         promotion && ((this.promotion = promotion), (this.lan += promotion)));
71 |     }
72 |     isCapture() {
73 |         return this.flags.indexOf(FLAGS.CAPTURE) > -1;
74 |     }
75 |     isPromotion() {
76 |         return this.flags.indexOf(FLAGS.PROMOTION) > -1;
77 |     }
78 |     isEnPassant() {
79 |         return this.flags.indexOf(FLAGS.EP_CAPTURE) > -1;
80 |     }
81 |     isKingsideCastle() {
82 |         return this.flags.indexOf(FLAGS.KSIDE_CASTLE) > -1;
83 |     }
84 |     isQueensideCastle() {
85 |         return this.flags.indexOf(FLAGS.QSIDE_CASTLE) > -1;
86 |     }
87 |     isBigPawn() {
88 |         return this.flags.indexOf(FLAGS.BIG_PAWN) > -1;
89 |     }
90 | }
91 | const FLAGS = {

```

```

92 | | | | NORMAL: 'n',
93 | | | | CAPTURE: 'c',
94 | | | | BIG_PAWN: 'b',
95 | | | | EP_CAPTURE: 'e',
96 | | | | PROMOTION: 'p',
97 | | | | KSIDE_CASTLE: 'k',
98 | | | | QSIDE_CASTLE: 'q',
99 | | | | NULL_MOVE: '-',
100 | | | },
101 | | BITS = {
102 | | | | NORMAL: 1,
103 | | | | CAPTURE: 2,
104 | | | | BIG_PAWN: 4,
105 | | | | EP_CAPTURE: 8,
106 | | | | PROMOTION: 16,
107 | | | | KSIDE_CASTLE: 32,
108 | | | | QSIDE_CASTLE: 64,
109 | | | | NULL_MOVE: 128,
110 | | | },
111 | | 0x88 = {
112 | | | | a8: 0,
113 | | | | b8: 1,
114 | | | | c8: 2,
115 | | | | d8: 3,
116 | | | | e8: 4,
117 | | | | f8: 5,
118 | | | | g8: 6,
119 | | | | h8: 7,
120 | | | | a7: 16,
121 | | | | b7: 17,
122 | | | | c7: 18,
123 | | | | d7: 19,
124 | | | | e7: 20,
125 | | | | f7: 21,
126 | | | | g7: 22,
127 | | | | h7: 23,
128 | | | | a6: 32,
129 | | | | b6: 33,
130 | | | | c6: 34,
131 | | | | d6: 35,
132 | | | | e6: 36,
133 | | | | f6: 37,
134 | | | | g6: 38,
135 | | | | h6: 39,
136 | | | | a5: 48,
137 | | | | b5: 49,
138 | | | | c5: 50,
139 | | | | d5: 51,
140 | | | | e5: 52,
141 | | | | f5: 53,
142 | | | | g5: 54,
143 | | | | h5: 55,
144 | | | | a4: 64,
145 | | | | b4: 65,
146 | | | | c4: 66,
147 | | | | d4: 67,
148 | | | | e4: 68,
149 | | | | f4: 69,
150 | | | | g4: 70,
151 | | | | h4: 71,
152 | | | | a3: 80,
153 | | | | b3: 81,
154 | | | | c3: 82,
155 | | | | d3: 83,
156 | | | | e3: 84,
157 | | | | f3: 85,
158 | | | | g3: 86,
159 | | | | h3: 87,
160 | | | | a2: 96,
161 | | | | b2: 97,
162 | | | | c2: 98,

```

```

163 | | | | | ·d2: 99,
164 | | | | | ·e2: 100,
165 | | | | | ·f2: 101,
166 | | | | | ·g2: 102,
167 | | | | | ·h2: 103,
168 | | | | | ·a1: 112,
169 | | | | | ·b1: 113,
170 | | | | | ·c1: 114,
171 | | | | | ·d1: 115,
172 | | | | | ·e1: 116,
173 | | | | | ·f1: 117,
174 | | | | | ·g1: 118,
175 | | | | | ·h1: 119,
176 | | | | | },
177 | | | | | ·PAWN_OFFSETS = { ·b: [16, 32, 17, 15], ·w: [-16, -32, -17, -15] },
178 | | | | | ·PIECE_OFFSETS = {
179 | | | | | ·n: [-18, -33, -31, -14, 18, 33, 31, 14],
180 | | | | | ·b: [-17, -15, 17, 15],
181 | | | | | ·r: [-16, 1, 16, -1],
182 | | | | | ·q: [-17, -16, -15, 1, 17, 16, 15, -1],
183 | | | | | ·k: [-17, -16, -15, 1, 17, 16, 15, -1],
184 | | | | | },
185 | | | | | ·ATTACKS = [
186 | | | | | ·20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0,
187 | | | | | ·20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0,
188 | | | | | ·0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2,
189 | | | | | ·24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 24, 24, 24,
190 | | | | | ·24, 24, 24, 56, 0, 56, 24, 24, 24, 24, 24, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0,
191 | | | | | ·0, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0,
192 | | | | | ·20, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0,
193 | | | | | ·24, 0, 0, 0, 0, 20, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0,
194 | | | | | ·0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20,
195 | | | | | ],
196 | | | | | ·RAYS = [
197 | | | | | ·17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0,
198 | | | | | ·15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0,
199 | | | | | ·0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0,
200 | | | | | ·16, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1,
201 | | | | | ·1, 1, 1, 0, -1, -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0,
202 | | | | | ·0, 0, 0, 0, 0, 0, 0, -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0,
203 | | | | | ·0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, -15, 0,
204 | | | | | ·0, 0, 0, -16, 0, 0, 0, 0, -17, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17, 0,
205 | | | | | ·0, -15, 0, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, 0, -17,
206 | | | | | ],
207 | | | | | ·PIECE_MASKS = { ·p: 1, ·n: 2, ·b: 4, ·r: 8, ·q: 16, ·k: 32 },
208 | | | | | ·PROMOTIONS = [ 'n', 'b', 'r', 'q' ],
209 | | | | | ·ROOKS = {
210 | | | | | ·w: [
211 | | | | | ·{ ·square: 0x88.a1, ·flag: BITS.QSIDE_CASTLE },
212 | | | | | ·{ ·square: 0x88.h1, ·flag: BITS.KSIDE_CASTLE },
213 | | | | | ],
214 | | | | | ·b: [
215 | | | | | ·{ ·square: 0x88.a8, ·flag: BITS.QSIDE_CASTLE },
216 | | | | | ·{ ·square: 0x88.h8, ·flag: BITS.KSIDE_CASTLE },
217 | | | | | ],
218 | | | | | },
219 | | | | | ·SECOND_RANK = { ·b: 1, ·w: 6 };
220 | | | | | function rank(square) {
221 | | | | | ·return square >> 4;
222 | | | | | }
223 | | | | | function file(square) {
224 | | | | | ·return 15 & square;
225 | | | | | }
226 | | | | | function isDigit(c) {
227 | | | | | ·return -1 ≠ '0123456789'.indexOf(c);
228 | | | | | }
229 | | | | | function algebraic(square) {
230 | | | | | ·const f = file(square),
231 | | | | | ·r = rank(square);
232 | | | | | ·return 'abcdefgh'.substring(f, f+1) + '87654321'.substring(r, r+1);
233 | | | | | }

```

```

234 |     function swapColor(color) {
235 |         return 'w' === color ? 'b' : 'w';
236 |     }
237 |     function validateFen(fen) {
238 |         const tokens = fen.split(/\s+/);
239 |         if (6 !== tokens.length)
240 |             return { ok: !1, error: 'Invalid FEN: must contain six space-delimited fields' };
241 |         const moveNumber = parseInt(tokens[5], 10);
242 |         if (isNaN(moveNumber) || moveNumber ≤ 0)
243 |             return { ok: !1, error: 'Invalid FEN: move number must be a positive integer' };
244 |         const halfMoves = parseInt(tokens[4], 10);
245 |         if (isNaN(halfMoves) || halfMoves < 0)
246 |             return {
247 |                 ok: !1,
248 |                 error: 'Invalid FEN: half-move counter number must be a non-negative integer',
249 |             };
250 |         if (!/^(?![abcdefgh][36])$/.test(tokens[3]))
251 |             return { ok: !1, error: 'Invalid FEN: en-passant square is invalid' };
252 |         if (/^[kKqQ-]$/ .test(tokens[2]))
253 |             return { ok: !1, error: 'Invalid FEN: castling availability is invalid' };
254 |         if (!/^(w|b)$/ .test(tokens[1]))
255 |             return { ok: !1, error: 'Invalid FEN: side-to-move is invalid' };
256 |         const rows = tokens[0].split('/');
257 |         if (8 !== rows.length)
258 |             return { ok: !1, error: 'Invalid FEN: piece data does not contain 8 '/'-delimited rows' };
259 |         for (let i = 0; i < rows.length; i++) {
260 |             let sumFields = 0,
261 |                 previousWasNumber = !1;
262 |             for (let k = 0; k < rows[i].length; k++)
263 |                 if (isDigit(rows[i][k])) {
264 |                     if (previousWasNumber)
265 |                         return { ok: !1, error: 'Invalid FEN: piece data is invalid (consecutive number)' };
266 |                     ((sumFields += parseInt(rows[i][k], 10)), (previousWasNumber = !0));
267 |                 } else {
268 |                     if (!/^[prnbqkPRNBQK]$/ .test(rows[i][k]))
269 |                         return { ok: !1, error: 'Invalid FEN: piece data is invalid (invalid piece)' };
270 |                     ((sumFields += 1), (previousWasNumber = !1));
271 |                 }
272 |             if (8 !== sumFields)
273 |                 return { ok: !1, error: 'Invalid FEN: piece data is invalid (too many squares in rank)' };
274 |         }
275 |         if (('3' === tokens[3][1] && 'w' === tokens[1]) || ('6' === tokens[3][1] && 'b' === tokens[1]))
276 |             return { ok: !1, error: 'Invalid FEN: illegal en-passant square' };
277 |         const kings = [
278 |             { color: 'white', regex: /K/g },
279 |             { color: 'black', regex: /k/g },
280 |         ];
281 |         for (const { color: color, regex: regex } of kings) {
282 |             if (!regex.test(tokens[0])) return { ok: !1, error: `Invalid FEN: missing ${color} king` };
283 |             if ((tokens[0].match(regex) || []).length > 1)
284 |                 return { ok: !1, error: `Invalid FEN: too many ${color} kings` };
285 |         }
286 |         return Array.from(rows[0] + rows[7]).some((char) => 'P' === char.toUpperCase())
287 |             ? { ok: !1, error: 'Invalid FEN: some pawns are on the edge rows' }
288 |             : { ok: !0 };
289 |     }
290 |     function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
291 |         const r = rank(to);
292 |         if ('p' !== piece || (7 !== r && 0 !== r))
293 |             moves.push({
294 |                 color: color,
295 |                 from: from,
296 |                 to: to,
297 |                 piece: piece,
298 |                 captured: captured,
299 |                 flags: flags,
300 |             });
301 |         else
302 |             for (let i = 0; i < PROMOTIONS.length; i++) {
303 |                 const promotion = PROMOTIONS[i];
304 |                 moves.push({

```

```

305 |         color: color,
306 |         from: from,
307 |         to: to,
308 |         piece: piece,
309 |         captured: captured,
310 |         promotion: promotion,
311 |         flags: flags | BITS.PROMOTION,
312 |     });
313 | }
314 | }
315 | return (
316 |     (exports.BISHOP = 'b'),
317 |     (exports.BLACK = 'b'),
318 |     (exports.Chess = class Chess {
319 |         _board = new Array(128);
320 |         _turn = 'w';
321 |         _kings = { w: -1, b: -1 };
322 |         _epSquare = -1;
323 |         _halfMoves = 0;
324 |         _moveNumber = 0;
325 |         _history = [];
326 |         _castling = { w: 0, b: 0 };
327 |         _hash = 0n;
328 |         _positionCount = new Map();
329 |         constructor(fen = DEFAULT_POSITION, { skipValidation: skipValidation = !1 } = {}) {
330 |             this.load(fen, { skipValidation: skipValidation });
331 |         }
332 |         clear() {
333 |             ((this._board = new Array(128)),
334 |             (this._kings = { w: -1, b: -1 }),
335 |             (this._turn = 'w'),
336 |             (this._castling = { w: 0, b: 0 }),
337 |             (this._epSquare = -1),
338 |             (this._halfMoves = 0),
339 |             (this._moveNumber = 1),
340 |             (this._history = []),
341 |             (this._hash = this._computeHash()),
342 |             (this._positionCount = new Map()));
343 |         }
344 |         load(fen, { skipValidation: skipValidation = !1 } = {}) {
345 |             let tokens = fen.split(/\s+/);
346 |             if (tokens.length ≥ 2 && tokens.length < 6) {
347 |                 const adjustments = ['- ', '- ', '0 ', '1 '];
348 |                 fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
349 |             }
350 |             if (((tokens = fen.split(/\s+/)), !skipValidation)) {
351 |                 const { ok: ok, error: error } = validateFen(fen);
352 |                 if (!ok) throw new Error(error);
353 |             }
354 |             const position = tokens[0];
355 |             let square = 0;
356 |             this.clear();
357 |             for (let i = 0; i < position.length; i++) {
358 |                 const piece = position.charAt(i);
359 |                 if ('/ ' === piece) square += 8;
360 |                 else if (isDigit(piece)) square += parseInt(piece, 10);
361 |                 else {
362 |                     const color = piece < 'a' ? 'w' : 'b';
363 |                     (this._put({ type: piece.toLowerCase(), color: color }, algebraic(square)), square++);
364 |                 }
365 |             }
366 |             ((this._turn = tokens[1]),
367 |             tokens[2].indexOf('K') > -1 && (this._castling.w |= BITS.KSIDE_CASTLE),
368 |             tokens[2].indexOf('Q') > -1 && (this._castling.w |= BITS.QSIDE_CASTLE),
369 |             tokens[2].indexOf('k') > -1 && (this._castling.b |= BITS.KSIDE_CASTLE),
370 |             tokens[2].indexOf('q') > -1 && (this._castling.b |= BITS.QSIDE_CASTLE),
371 |             (this._epSquare = '-' === tokens[3] ? -1 : 0x88[tokens[3]]),
372 |             (this._halfMoves = parseInt(tokens[4], 10)),
373 |             (this._moveNumber = parseInt(tokens[5], 10)),
374 |             (this._hash = this._computeHash()),
375 |             this._incPositionCount());

```

```

376 |         }
377 |         fen({ forceEnpassantSquare: forceEnpassantSquare !== 1 } !== {} ) {
378 |             let empty = 0,
379 |             fen = '';
380 |             for (let i = 0x88.a8; i ≤ 0x88.h1; i++) {
381 |                 if (this._board[i]) {
382 |                     empty > 0 && ((fen += empty), (empty = 0));
383 |                     const { color: color, type: piece } = this._board[i];
384 |                     fen += 'w' === color ? piece.toUpperCase() : piece.toLowerCase();
385 |                 } else empty++;
386 |                 (i + 1) & 136 &&
387 |                 ((empty > 0 && (fen += empty), i !== 0x88.h1 && (fen += '/'), (empty = 0), (i += 8)));
388 |             }
389 |             let castling = '';
390 |             (this._castling.w & BITS.KSIDE_CASTLE && (castling += 'K'),
391 |             this._castling.w & BITS.QSIDE_CASTLE && (castling += 'Q'),
392 |             this._castling.b & BITS.KSIDE_CASTLE && (castling += 'k'),
393 |             this._castling.b & BITS.QSIDE_CASTLE && (castling += 'q'),
394 |             (castling = castling || '-'));
395 |             let epSquare = '-';
396 |             if (-1 !== this._epSquare)
397 |                 if (forceEnpassantSquare) epSquare = algebraic(this._epSquare);
398 |             else {
399 |                 const bigPawnSquare = this._epSquare + ('w' === this._turn ? 16 : -16),
400 |                 squares = [bigPawnSquare + 1, bigPawnSquare - 1];
401 |                 for (const square of squares) {
402 |                     if (136 & square) continue;
403 |                     const color = this._turn;
404 |                     if (this._board[square]?.color === color && 'p' === this._board[square]?.type) {
405 |                         this._makeMove({
406 |                             color: color,
407 |                             from: square,
408 |                             to: this._epSquare,
409 |                             piece: 'p',
410 |                             captured: 'p',
411 |                             flags: BITS.EP_CAPTURE,
412 |                         });
413 |                         const isLegal = !this._isKingAttacked(color);
414 |                         if ((this._undoMove(), isLegal)) {
415 |                             epSquare = algebraic(this._epSquare);
416 |                             break;
417 |                         }
418 |                     }
419 |                 }
420 |             }
421 |             return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
422 |         }
423 |         _pieceKey(i) {
424 |             if (!this._board[i]) return 0n;
425 |             const { color: color, type: type } = this._board[i],
426 |             typeIndex = { p: 0, n: 1, b: 2, r: 3, q: 4, k: 5 }[type];
427 |             return PIECE_KEYS[{ w: 0, b: 1 }[color]][typeIndex][i];
428 |         }
429 |         _epKey() {
430 |             return -1 === this._epSquare ? 0n : EP_KEYS[7 & this._epSquare];
431 |         }
432 |         _castlingKey() {
433 |             const index = (this._castling.w >> 5) | (this._castling.b >> 3);
434 |             return CASTLING_KEYS[index];
435 |         }
436 |         _computeHash() {
437 |             let hash = 0n;
438 |             for (let i = 0x88.a8; i ≤ 0x88.h1; i++)
439 |                 136 & i ? (i += 7) : this._board[i] && (hash ^= this._pieceKey(i));
440 |             return (
441 |                 (hash ^= this._epKey()),
442 |                 (hash ^= this._castlingKey()),
443 |                 'b' === this._turn && (hash ^= SIDE_KEY),
444 |                 hash
445 |             );
446 |         }

```

```

447 |         .reset()·{
448 |             .this.load(DEFAULT_POSITION);
449 |         }
450 |         .get(square)·{
451 |             .return·this._board[0x88[square]];
452 |         }
453 |         .set(sq,·piece)·{
454 |             .((this._hash·^=·this._pieceKey(sq)),
455 |             .(this._board[sq]·:=·piece),
456 |             .(this._hash·^=·this._pieceKey(sq)));
457 |         }
458 |         .put({·type:·type,·color:·color·},·square)·{
459 |             .if·(-1·===·'pnbrqkPNBRQK'.indexOf(type.toLowerCase()))·return·!1;
460 |             .if·(!(square·in·0x88))·return·!1;
461 |             .const·sq·:=·0x88[square];
462 |             .if·('k'·===·type·&&·-1·≠·this._kings[color]·&&·this._kings[color]·≠·sq)·return·!1;
463 |             .const·currentPieceOnSquare·:=·this._board[sq];
464 |             .return·(
465 |                 .currentPieceOnSquare·&&
466 |                 .('k'·===·currentPieceOnSquare.type·&&
467 |                 .(this._kings[currentPieceOnSquare.color]·:=·-1),
468 |                 .this._set(sq,·{·type:·type,·color:·color·}),
469 |                 .('k'·===·type·&&·(this._kings[color]·:=·sq),
470 |                 .!0
471 |             .);
472 |         }
473 |         .clear(sq)·{
474 |             .((this._hash·^=·this._pieceKey(sq)),·delete·this._board[sq]);
475 |         }
476 |         .attacked(color,·square,·verbose)·{
477 |             .const·attackers·:=·[];
478 |             .for·(let·i·:=·0x88.a8;·i·≤·0x88.h1;·i++)·{
479 |                 .if·(136·&·i)·{
480 |                     .i·+=·7;
481 |                     .continue;
482 |                 }
483 |                 .if·(void·0·===·this._board[i]·||·this._board[i].color·≠·color)·continue;
484 |                 .const·piece·:=·this._board[i],
485 |                 .difference·:=·i·-·square;
486 |                 .if·(0·===·difference)·continue;
487 |                 .const·index·:=·difference·+·119;
488 |                 .if·(ATTACKS[index]·&·PIECE_MASKS[piece.type])·{
489 |                     .if·('p'·===·piece.type)·{
490 |                         .if·(
491 |                             .(difference·>·0·&&·'w'·===·piece.color)·||
492 |                             .(difference·≤·0·&&·'b'·===·piece.color)
493 |                         )·{
494 |                             .if·(!verbose)·return·!0;
495 |                             .attackers.push(algebraic(i));
496 |                         }
497 |                         .continue;
498 |                     }
499 |                     .if·('n'·===·piece.type·||·'k'·===·piece.type)·{
500 |                         .if·(verbose)·{
501 |                             .attackers.push(algebraic(i));
502 |                             .continue;
503 |                         }
504 |                         .return·!0;
505 |                     }
506 |                     .const·offset·:=·RAYS[index];
507 |                     .let·j·:=·i·+·offset,
508 |                     .blocked·:=·!1;
509 |                     .for·(;·j·≠·square;·){
510 |                         .if·(null·≠·this._board[j])·{
511 |                             .blocked·:=·!0;
512 |                             .break;
513 |                         }
514 |                         .j·+=·offset;
515 |                     }
516 |                     .if·(!blocked)·{
517 |                         .if·(verbose)·{

```

```

518 |         attackers.push(algebraic(i));
519 |         continue;
520 |     }
521 |     return !0;
522 | }
523 | }
524 | }
525 |     return !!verbose && attackers;
526 | }
527 |     attackers(square, attackedBy) {
528 |         return attackedBy
529 |             ? this._attacked(attackedBy, 0x88[square], !0)
530 |             : this._attacked(this._turn, 0x88[square], !0);
531 |     }
532 |     _isKingAttacked(color) {
533 |         const square = this._kings[color];
534 |         return -1 & square && this._attacked(swapColor(color), square);
535 |     }
536 |     hash() {
537 |         return this._hash.toString(16);
538 |     }
539 |     isAttacked(square, attackedBy) {
540 |         return this._attacked(attackedBy, 0x88[square]);
541 |     }
542 |     isCheck() {
543 |         return this._isKingAttacked(this._turn);
544 |     }
545 |     inCheck() {
546 |         return this.isCheck();
547 |     }
548 |     isCheckmate() {
549 |         return this.isCheck() && 0 === this._moves().length;
550 |     }
551 |     isStalemate() {
552 |         return !this.isCheck() && 0 === this._moves().length;
553 |     }
554 |     isInsufficientMaterial() {
555 |         const pieces = {b: 0, n: 0, r: 0, q: 0, k: 0, p: 0},
556 |             bishops = [];
557 |         let numPieces = 0,
558 |             squareColor = 0;
559 |         for (let i = 0x88.a8; i <= 0x88.h1; i++) {
560 |             if (((squareColor = (squareColor + 1) % 2), 136 & i)) {
561 |                 i += 7;
562 |                 continue;
563 |             }
564 |             const piece = this._board[i];
565 |             piece &&
566 |                 ((pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1),
567 |                 'b' === piece.type && bishops.push(squareColor),
568 |                 numPieces++);
569 |         }
570 |         if (2 === numPieces) return !0;
571 |         if (3 === numPieces && (1 === pieces.b || 1 === pieces.n)) return !0;
572 |         if (numPieces === pieces.b + 2) {
573 |             let sum = 0;
574 |             const len = bishops.length;
575 |             for (let i = 0; i < len; i++) sum += bishops[i];
576 |             if (0 === sum || sum === len) return !0;
577 |         }
578 |         return !1;
579 |     }
580 |     isThreefoldRepetition() {
581 |         return this._getPositionCount(this._hash) >= 3;
582 |     }
583 |     isDrawByFiftyMoves() {
584 |         return this._halfMoves >= 100;
585 |     }
586 |     isDraw() {
587 |         return (
588 |             this.isDrawByFiftyMoves() ||

```



```

589 |         .this.isStalemate() ||
590 |         .this.isInsufficientMaterial() ||
591 |         .this.isThreefoldRepetition()
592 |     );
593 | }
594 | .isGameOver() {
595 |     return .this.isCheckmate() || .this.isDraw();
596 | }
597 | .moves({ .verbose: .verbose = !1, .square: .square, .piece: .piece } = {}) {
598 |     const moves = .this._moves({ .square: .square, .piece: .piece });
599 |     return .verbose
600 |         ? .moves.map((move) => new Move(this, move))
601 |         : .moves.map((move) => this._moveToSan(move, moves));
602 | }
603 | .moves({ .legal: .legal = !0, .piece: .piece, .square: .square } = {}) {
604 |     const forSquare = .square ? .square.toLowerCase() : .void 0,
605 |           forPiece = .piece ? .toLowerCase(),
606 |           moves = [],
607 |           us = this._turn,
608 |           them = swapColor(us);
609 |     let firstSquare = 0x88.a8,
610 |         lastSquare = 0x88.h1,
611 |         singleSquare = !1;
612 |     if (forSquare) {
613 |         if (!forSquare in 0x88) return [];
614 |         ((firstSquare = lastSquare = 0x88[forSquare]), (singleSquare = !0));
615 |     }
616 |     for (let from = firstSquare; from <= lastSquare; from++) {
617 |         if (136 & from) {
618 |             from += 7;
619 |             continue;
620 |         }
621 |         if (!this._board[from] || this._board[from].color == them) continue;
622 |         const { type: type } = this._board[from];
623 |         let to;
624 |         if ('p' == type) {
625 |             if (forPiece & forPiece != type) continue;
626 |             ((to = from + PAWN_OFFSETS[us][0]),
627 |              this._board[to] ||
628 |              (addMove(moves, us, from, to, 'p'),
629 |               (to = from + PAWN_OFFSETS[us][1]),
630 |               SECOND_RANK[us] != rank(from) ||
631 |               this._board[to] ||
632 |               addMove(moves, us, from, to, 'p', .void 0, BITS.BIG_PAWN))));
633 |             for (let j = 2; j < 4; j++)
634 |                 ((to = from + PAWN_OFFSETS[us][j]),
635 |                  136 & to ||
636 |                  (this._board[to] ? .color == them
637 |                   ? addMove(moves, us, from, to, 'p', this._board[to].type, BITS.CAPTURE)
638 |                   : to == this._epSquare &
639 |                     addMove(moves, us, from, to, 'p', 'p', BITS.EP_CAPTURE)));
640 |             } else {
641 |                 if (forPiece & forPiece != type) continue;
642 |                 for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
643 |                     const offset = PIECE_OFFSETS[type][j];
644 |                     for (to = from; (to += offset), !(136 & to);) {
645 |                         if (this._board[to]) {
646 |                             if (this._board[to].color == us) break;
647 |                             addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
648 |                             break;
649 |                         }
650 |                         if ((addMove(moves, us, from, to, type), 'n' == type || 'k' == type)) break;
651 |                     }
652 |                 }
653 |             }
654 |         }
655 |         if (
656 |             !(
657 |                 (void 0 != forPiece & 'k' != forPiece) ||
658 |                 (singleSquare & lastSquare != this._kings[us])
659 |             )

```

```

660 |         }
661 |         if (this._castling[us] & BITS.KSIDE_CASTLE) {
662 |             const castlingFrom = this._kings[us],
663 |                   castlingTo = castlingFrom + 2;
664 |             this._board[castlingFrom + 1] ||
665 |             this._board[castlingTo] ||
666 |             this._attacked(them, this._kings[us]) ||
667 |             this._attacked(them, castlingFrom + 1) ||
668 |             this._attacked(them, castlingTo) ||
669 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.KSIDE_CASTLE);
670 |         }
671 |         if (this._castling[us] & BITS.QSIDE_CASTLE) {
672 |             const castlingFrom = this._kings[us],
673 |                   castlingTo = castlingFrom - 2;
674 |             this._board[castlingFrom - 1] ||
675 |             this._board[castlingFrom - 2] ||
676 |             this._board[castlingFrom - 3] ||
677 |             this._attacked(them, this._kings[us]) ||
678 |             this._attacked(them, castlingFrom - 1) ||
679 |             this._attacked(them, castlingTo) ||
680 |             addMove(moves, us, this._kings[us], castlingTo, 'k', void 0, BITS.QSIDE_CASTLE);
681 |         }
682 |     }
683 |     if (!legal || --1 === this._kings[us]) return moves;
684 |     const legalMoves = [];
685 |     for (let i = 0, len = moves.length; i < len; i++)
686 |         (this._makeMove(moves[i]),
687 |          this._isKingAttacked(us) || legalMoves.push(moves[i]),
688 |          this._undoMove());
689 |     return legalMoves;
690 | }
691 | move(move) {
692 |     let moveObj = null;
693 |     const moves = this._moves();
694 |     for (let i = 0, len = moves.length; i < len; i++)
695 |         if (
696 |             move.from === algebraic(moves[i].from) &&
697 |             move.to === algebraic(moves[i].to) &&
698 |             (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
699 |         ) {
700 |             moveObj = moves[i];
701 |             break;
702 |         }
703 |     if (!moveObj) throw new Error(`Invalid move: ${JSON.stringify(move)}`);
704 |     const prettyMove = new Move(this, moveObj);
705 |     return (this._makeMove(moveObj), this._incPositionCount(), prettyMove);
706 | }
707 | _push(move) {
708 |     this._history.push({
709 |         move: move,
710 |         kings: {b: this._kings.b, w: this._kings.w},
711 |         turn: this._turn,
712 |         castling: {b: this._castling.b, w: this._castling.w},
713 |         epSquare: this._epSquare,
714 |         halfMoves: this._halfMoves,
715 |         moveNumber: this._moveNumber,
716 |     });
717 | }
718 | _movePiece(from, to) {
719 |     ((this._hash ^= this._pieceKey(from)),
720 |      (this._board[to] = this._board[from]),
721 |      delete this._board[from],
722 |      (this._hash ^= this._pieceKey(to)));
723 | }
724 | _makeMove(move) {
725 |     const us = this._turn,
726 |           them = swapColor(us);
727 |     if ((this._push(move), move.flags & BITS.NULL_MOVE))
728 |         return (
729 |             'b' === us && this._moveNumber++,
730 |             this._halfMoves++,

```

```

731 |         (this._turn = them),
732 |         void (this._epSquare = -1)
733 |     );
734 |     if (
735 |         ((this._hash ^ this._epKey()),
736 |         (this._hash ^ this._castlingKey()),
737 |         move.captured & (this._hash ^ this._pieceKey(move.to)),
738 |         this._movePiece(move.from, move.to),
739 |         move.flags & BITS.EP_CAPTURE &
740 |         ('b' === this._turn ? this._clear(move.to - 16) : this._clear(move.to + 16)),
741 |         move.promotion &
742 |         (this._clear(move.to), this._set(move.to, { type: move.promotion, color: us })),
743 |         'k' === this._board[move.to].type)
744 |     ) {
745 |         if (((this._kings[us] = move.to), move.flags & BITS.KSIDE_CASTLE)) {
746 |             const castlingTo = move.to - 1,
747 |                 castlingFrom = move.to + 1;
748 |             this._movePiece(castlingFrom, castlingTo);
749 |         } else if (move.flags & BITS.QSIDE_CASTLE) {
750 |             const castlingTo = move.to + 1,
751 |                 castlingFrom = move.to - 2;
752 |             this._movePiece(castlingFrom, castlingTo);
753 |         }
754 |         this._castling[us] = 0;
755 |     }
756 |     if (this._castling[us])
757 |         for (let i = 0, len = ROOKS[us].length; i < len; i++)
758 |             if (move.from === ROOKS[us][i].square & this._castling[us] & ROOKS[us][i].flag) {
759 |                 this._castling[us] ^= ROOKS[us][i].flag;
760 |                 break;
761 |             }
762 |     if (this._castling[them])
763 |         for (let i = 0, len = ROOKS[them].length; i < len; i++)
764 |             if (move.to === ROOKS[them][i].square & this._castling[them] & ROOKS[them][i].flag) {
765 |                 this._castling[them] ^= ROOKS[them][i].flag;
766 |                 break;
767 |             }
768 |     if (((this._hash ^ this._castlingKey()), move.flags & BITS.BIG_PAWN)) {
769 |         let epSquare;
770 |         ((epSquare = 'b' === us ? move.to - 16 : move.to + 16),
771 |         ((move.to - 1) & 136 ||
772 |         'p' !== this._board[move.to - 1] ? type :
773 |         this._board[move.to - 1] ? color : them) &
774 |         ((move.to + 1) & 136 ||
775 |         'p' !== this._board[move.to + 1] ? type :
776 |         this._board[move.to + 1] ? color : them)
777 |         ? (this._epSquare = -1)
778 |         : ((this._epSquare = epSquare), (this._hash ^ this._epKey())));
779 |     } else this._epSquare = -1;
780 |     ('p' === move.piece || move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)
781 |     ? (this._halfMoves = 0)
782 |     : this._halfMoves++,
783 |     'b' === us & this._moveNumber++,
784 |     (this._turn = them),
785 |     (this._hash ^ SIDE_KEY));
786 | }
787 | undo() {
788 |     const hash = this._hash,
789 |         move = this._undoMove();
790 |     if (move) {
791 |         const prettyMove = new Move(this, move);
792 |         return (this._decPositionCount(hash), prettyMove);
793 |     }
794 |     return null;
795 | }
796 | _undoMove() {
797 |     const old = this._history.pop();
798 |     if (void 0 === old) return null;
799 |     ((this._hash ^ this._epKey()), (this._hash ^ this._castlingKey()));
800 |     const move = old.move;
801 |     ((this._kings = old.kings),

```

```

802 |         (this._turn = old.turn),
803 |         (this._castling = old.castling),
804 |         (this._epSquare = old.epSquare),
805 |         (this._halfMoves = old.halfMoves),
806 |         (this._moveNumber = old.moveNumber),
807 |         (this._hash ^= this._epKey()),
808 |         (this._hash ^= this._castlingKey()),
809 |         (this._hash ^= SIDE_KEY));
810 |     const us = this._turn,
811 |           them = swapColor(us);
812 |     if (move.flags & BITS.NULL_MOVE) return move;
813 |     if (
814 |         (this._movePiece(move.to, move.from),
815 |         move.piece &&
816 |         (this._clear(move.from), this._set(move.from, { type: move.piece, color: us })),
817 |         move.captured)
818 |     )
819 |         if (move.flags & BITS.EP_CAPTURE) {
820 |             let index;
821 |             ((index = 'b' === us ? move.to - 16 : move.to + 16),
822 |             this._set(index, { type: 'p', color: them }));
823 |         } else this._set(move.to, { type: move.captured, color: them });
824 |     if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
825 |         let castlingTo, castlingFrom;
826 |         (move.flags & BITS.KSIDE_CASTLE
827 |         ? ((castlingTo = move.to + 1), (castlingFrom = move.to - 1))
828 |         : ((castlingTo = move.to - 2), (castlingFrom = move.to + 1)),
829 |         this._movePiece(castlingFrom, castlingTo));
830 |     }
831 |     return move;
832 | }
833 | _moveToSan(move, moves) {
834 |     let output = '';
835 |     if (move.flags & BITS.KSIDE_CASTLE) output += 'O-O';
836 |     else if (move.flags & BITS.QSIDE_CASTLE) output += 'O-O-O';
837 |     else {
838 |         if ('p' !== move.piece) {
839 |             const disambiguator = (function getDisambiguator(move, moves) {
840 |                 const from = move.from,
841 |                       to = move.to,
842 |                       piece = move.piece;
843 |                 let ambiguities = 0,
844 |                     sameRank = 0,
845 |                     sameFile = 0;
846 |                 for (let i = 0, len = moves.length; i < len; i++) {
847 |                     const ambigFrom = moves[i].from,
848 |                           ambigTo = moves[i].to;
849 |                     piece === moves[i].piece &&
850 |                     from !== ambigFrom &&
851 |                     to === ambigTo &&
852 |                     (ambiguities++,
853 |                     rank(from) === rank(ambigFrom) && sameRank++,
854 |                     file(from) === file(ambigFrom) && sameFile++);
855 |                 }
856 |                 return ambiguities > 0
857 |                     ? sameRank > 0 && sameFile > 0
858 |                     : ? algebraic(from)
859 |                       : sameFile > 0
860 |                       ? algebraic(from).charAt(1)
861 |                       : algebraic(from).charAt(0)
862 |                       : '';
863 |             })(move, moves);
864 |             output += move.piece.toUpperCase() + disambiguator;
865 |         }
866 |         if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE) &&
867 |         ('p' === move.piece && (output += algebraic(move.from)[0]), (output += 'x')),
868 |         (output += algebraic(move.to)),
869 |         move.promotion && (output += '=' + move.promotion.toUpperCase()));
870 |     }
871 |     return (
872 |         this._makeMove(move),

```

```

873 |         .this.isCheck() && (this.isCheckmate() ? (output += '#') : (output += '+')),
874 |         .this._undoMove(),
875 |         .output
876 |     );
877 | }
878 |     .turn() {
879 |         .return this._turn;
880 |     }
881 |     .board() {
882 |         .const output = [];
883 |         .let row = [];
884 |         .for (let i = 0x88.a8; i <= 0x88.h1; i++)
885 |             .(null == this._board[i]
886 |             ? row.push(null)
887 |             : row.push({
888 |                 .square: algebraic(i),
889 |                 .type: this._board[i].type,
890 |                 .color: this._board[i].color,
891 |             })),
892 |         .(i + 1) & 136 && (output.push(row), (row = []), (i += 8));
893 |         .return output;
894 |     }
895 |     .squareColor(square) {
896 |         .if (square in 0x88) {
897 |             .const sq = 0x88[square];
898 |             .return (rank(sq) + file(sq)) % 2 == 0 ? 'light' : 'dark';
899 |         }
900 |         .return null;
901 |     }
902 |     .history({ verbose: verbose == !1 } = {}) {
903 |         .const reversedHistory = [],
904 |             .moveHistory = [];
905 |         .for (; this._history.length > 0;) reversedHistory.push(this._undoMove());
906 |         .for (;;) {
907 |             .const move = reversedHistory.pop();
908 |             .if (!move) break;
909 |             .(verbose
910 |             ? moveHistory.push(new Move(this, move))
911 |             : moveHistory.push(this._moveToSan(move, this._moves()))),
912 |             .this._makeMove(move);
913 |         }
914 |         .return moveHistory;
915 |     }
916 |     ._getPositionCount(hash) {
917 |         .return this._positionCount.get(hash) ?? 0;
918 |     }
919 |     ._incPositionCount() {
920 |         .this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
921 |     }
922 |     ._decPositionCount(hash) {
923 |         .const currentCount = this._positionCount.get(hash) ?? 0;
924 |         .1 == currentCount
925 |         ? this._positionCount.delete(hash)
926 |         : this._positionCount.set(hash, currentCount - 1);
927 |     }
928 |     .moveNumber() {
929 |         .return this._moveNumber;
930 |     }
931 | },
932 | (exports.DEFAULT_POSITION = DEFAULT_POSITION),
933 | (exports.KING = 'k'),
934 | (exports.KNIGHT = 'n'),
935 | (exports.Move = Move),
936 | (exports.PAWN = 'p'),
937 | (exports.QUEEN = 'q'),
938 | (exports.ROOK = 'r'),
939 | (exports.SQUARES = [
940 |     'a8',
941 |     'b8',
942 |     'c8',
943 |     'd8',

```

```

944 | | | | | 'e8',
945 | | | | | 'f8',
946 | | | | | 'g8',
947 | | | | | 'h8',
948 | | | | | 'a7',
949 | | | | | 'b7',
950 | | | | | 'c7',
951 | | | | | 'd7',
952 | | | | | 'e7',
953 | | | | | 'f7',
954 | | | | | 'g7',
955 | | | | | 'h7',
956 | | | | | 'a6',
957 | | | | | 'b6',
958 | | | | | 'c6',
959 | | | | | 'd6',
960 | | | | | 'e6',
961 | | | | | 'f6',
962 | | | | | 'g6',
963 | | | | | 'h6',
964 | | | | | 'a5',
965 | | | | | 'b5',
966 | | | | | 'c5',
967 | | | | | 'd5',
968 | | | | | 'e5',
969 | | | | | 'f5',
970 | | | | | 'g5',
971 | | | | | 'h5',
972 | | | | | 'a4',
973 | | | | | 'b4',
974 | | | | | 'c4',
975 | | | | | 'd4',
976 | | | | | 'e4',
977 | | | | | 'f4',
978 | | | | | 'g4',
979 | | | | | 'h4',
980 | | | | | 'a3',
981 | | | | | 'b3',
982 | | | | | 'c3',
983 | | | | | 'd3',
984 | | | | | 'e3',
985 | | | | | 'f3',
986 | | | | | 'g3',
987 | | | | | 'h3',
988 | | | | | 'a2',
989 | | | | | 'b2',
990 | | | | | 'c2',
991 | | | | | 'd2',
992 | | | | | 'e2',
993 | | | | | 'f2',
994 | | | | | 'g2',
995 | | | | | 'h2',
996 | | | | | 'a1',
997 | | | | | 'b1',
998 | | | | | 'c1',
999 | | | | | 'd1',
1000 | | | | | 'e1',
1001 | | | | | 'f1',
1002 | | | | | 'g1',
1003 | | | | | 'h1',
1004 | | | | | ],
1005 | | | | | (exports.WHITE = 'w'),
1006 | | | | | (exports.validateFen = validateFen),
1007 | | | | | (exports.xoroshiro128 = xoroshiro128),
1008 | | | | | Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' }),
1009 | | | | | exports
1010 | | | | | );
1011 | | | | | })({});
1012 |
1013 | | return TSCSChessShared;
1014 | }

```

```

1015 | var TSCSessShared = __TSCSessCreateChessCore();
1016 | globalThis.TSCSessShared = TSCSessShared;
1017 | globalThis.__TSCSess = globalThis.__TSCSess || {};
1018 | globalThis.__TSCSess.factories = globalThis.__TSCSess.factories || {};
1019 | globalThis.__TSCSess.factories.chessCore = __TSCSessCreateChessCore;
1020 | globalThis.__TSCSess.chessCore = TSCSessShared;
1021 |

```

==== src/chess/chess-runtime.js (540 B / 958b2454f794) ====

```

1 | var __TSCSessRuntime = (function (exports) {
2 |   'use strict';
3 |   const sharedCore = globalThis.TSCSessShared;
4 |   if (!sharedCore) throw new Error('Missing shared chess runtime: TSCSessShared');
5 |   const Chess = sharedCore.Chess,
6 |     SQUARES = sharedCore.SQUARES;
7 |   return (
8 |     (exports.Chess = Chess),
9 |     (exports.SQUARES = SQUARES),
10 |    Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' }),
11 |    exports
12 |  );
13 | })({});
14 |
15 | globalThis.__TSCSess = globalThis.__TSCSess || {};
16 | globalThis.__TSCSess.chessRuntime = __TSCSessRuntime;
17 |

```

==== src/main.js (83 B / b963e36e3c82) ====

```

1 | !(function () {
2 |   'use strict';
3 |   globalThis.__TSCSess.app.createChessApp();
4 | })();
5 |

```

==== src/styles/main.css (13601 B / 8946cbef0b66) ====

```

1 | :root {
2 |   --app-padding: 28px;
3 |   --viewport-height: 100vh;
4 |   --page: oklch(95% 0.02 85deg / 1);
5 |   --ink: oklch(26% 0.01 250deg / 1);
6 |   --muted: oklch(54% 0.01 185deg / 1);
7 |   --line: oklch(81% 0.02 80deg / 1);
8 |   --panel: oklch(99% 0.01 85deg / 1);
9 |   --button: oklch(33% 0.03 205deg / 1);
10 |   --button-text: oklch(100% 0 0deg / 1);
11 |   --accent: oklch(55% 0.15 35deg / 1);
12 |   --light-square: oklch(89% 0.05 90deg / 1);
13 |   --dark-square: oklch(59% 0.08 150deg / 1);
14 |   --last-move: oklch(80% 0.12 90deg / 1);
15 |   --legal: oklch(43% 0.12 260deg / 1);
16 |   --capture: oklch(52% 0.15 25deg / 1);
17 |   --shadow: 0 18px 42px oklch(24% 0.01 75deg / 0.18);
18 |   --clear-color: oklch(0% 0 0deg / 0);
19 |   --panel-soft: oklch(99% 0.01 85deg / 0.68);
20 |   --panel-raised: oklch(99% 0.01 85deg / 0.74);
21 |   --panel-strong: oklch(99% 0.01 85deg / 0.88);
22 |   --board-edge: oklch(33% 0.03 155deg / 1);
23 |   --last-move-strong: oklch(80% 0.12 90deg / 0.76);
24 |   --last-move-soft: oklch(80% 0.12 90deg / 0.68);
25 |   --legal-marker: oklch(43% 0.12 260deg / 0.42);
26 |   --capture-marker: oklch(52% 0.15 25deg / 0.74);
27 |   --piece-shadow: oklch(0% 0 0deg / 0.2);
28 |   --active-ring: oklch(100% 0 0deg / 0.3);
29 |   --active-halo: oklch(33% 0.03 205deg / 0.18);
30 |   --warning: oklch(69% 0.15 75deg / 1);
31 |   --line-soft: oklch(81% 0.02 80deg / 0.58);
32 |   --line-firm: oklch(81% 0.02 80deg / 0.62);
33 |   --accent-soft: oklch(55% 0.15 35deg / 0.12);
34 |   --button-soft: oklch(33% 0.03 205deg / 0.08);
35 |   --overlay: oklch(21% 0.01 240deg / 0.58);
36 | }

```

```

37 |
38 | @supports (height: 100dvh) {
39 |   ::root {
40 |     --viewport-height: 100dvh;
41 |   }
42 | }
43 | [hidden] {
44 |   display: none !important;
45 | }
46 |
47 | .chess-app {
48 |   --board-min-size: 560px;
49 |   --board-vertical-breathing: 104px;
50 |   --side-columns-width: 628px;
51 |   --grid-gaps-width: 30px;
52 |   --board-size: max(
53 |     var(--board-min-size),
54 |     min(
55 |       calc(
56 |         var(--viewport-height) - var(--app-padding) - var(--app-padding) -
57 |         var(--board-vertical-breathing)
58 |       ),
59 |       calc(
60 |         100vw - var(--app-padding) - var(--app-padding) - var(--side-columns-width) -
61 |         var(--grid-gaps-width)
62 |       )
63 |     )
64 |   );
65 |   --panel-max-height: calc(var(--viewport-height) - var(--app-padding) - var(--app-padding));
66 |   width: 100%;
67 |   min-width: 1280px;
68 |   min-height: var(--viewport-height);
69 |   display: grid;
70 |   grid-template-columns: 230px var(--board-size) 118px 280px;
71 |   gap: 10px;
72 |   align-items: stretch;
73 |   justify-content: center;
74 |   overflow: hidden;
75 |   padding: var(--app-padding);
76 |   background: var(--page);
77 |   color: var(--ink);
78 | }
79 |
80 | .board-stage {
81 |   width: var(--board-size);
82 |   align-self: center;
83 |   justify-self: center;
84 | }
85 |
86 | .clock-panel {
87 |   width: 100%;
88 |   height: var(--board-size);
89 |   min-width: 0;
90 |   min-height: 0;
91 |   align-self: center;
92 |   justify-self: stretch;
93 |   display: flex;
94 | }
95 |
96 | .chess-board {
97 |   width: 100%;
98 |   aspect-ratio: 1;
99 |   display: grid;
100 |   grid-template-columns: repeat(8, minmax(0, 1fr));
101 |   overflow: hidden;
102 |   border: 3px solid var(--board-edge);
103 |   border-radius: 8px;
104 |   box-shadow: var(--shadow);
105 |   background: var(--board-edge);
106 | }
107 |

```



```

108 | .board-square {
109 |     position: relative;
110 |     width: 100%;
111 |     aspect-ratio: 1;
112 |     display: grid;
113 |     place-items: center;
114 |     min-width: 0;
115 |     padding: 0;
116 |     border: 0;
117 |     color: var(--ink);
118 |     transition:
119 |         box-shadow 140ms ease,
120 |         filter 140ms ease,
121 |         transform 140ms ease;
122 | }
123 |
124 | .board-square:not(:disabled):hover {
125 |     filter: brightness(1.05);
126 | }
127 |
128 | .board-square.is-light {
129 |     background: var(--light-square);
130 | }
131 |
132 | .board-square.is-dark {
133 |     background: var(--dark-square);
134 | }
135 |
136 | .board-square.is-selected {
137 |     box-shadow: inset 0 0 0 4px var(--legal);
138 | }
139 |
140 | .board-square.is-last-move {
141 |     box-shadow: inset 0 0 0 4px var(--last-move-strong);
142 | }
143 |
144 | .board-square.is-selected.is-last-move {
145 |     box-shadow:
146 |         inset 0 0 0 4px var(--legal),
147 |         inset 0 0 0 8px var(--last-move-soft);
148 | }
149 |
150 | .board-square.is-legal::after,
151 | .board-square.is-capture::after {
152 |     position: absolute;
153 |     z-index: 1;
154 |     content: '';
155 |     border-radius: 999px;
156 |     pointer-events: none;
157 | }
158 |
159 | .board-square.is-legal::after {
160 |     width: 28%;
161 |     height: 28%;
162 |     background: var(--legal-marker);
163 | }
164 |
165 | .board-square.is-capture::after {
166 |     inset: 12%;
167 |     border: 3px solid var(--capture-marker);
168 | }
169 |
170 | .board-square.is-check {
171 |     box-shadow: inset 0 0 0 5px var(--capture);
172 | }
173 |
174 | .chess-board.is-locked .board-square {
175 |     filter: saturate(0.86);
176 | }
177 |
178 | .piece {

```

```

179 | z-index: 2;
180 | display: block;
181 | font-size: 4rem;
182 | font-size: max(4rem, var(--board-size) / 8 * 0.92);
183 | line-height: 1;
184 | text-shadow: 0 2px 6px var(--piece-shadow);
185 | user-select: none;
186 | }
187 |
188 | .coordinate {
189 |   position: absolute;
190 |   z-index: 3;
191 |   font-size: 0.7rem;
192 |   font-size: max(0.7rem, var(--board-size) / 80);
193 |   font-weight: 800;
194 |   line-height: 1;
195 |   opacity: 0.72;
196 |   pointer-events: none;
197 | }
198 |
199 | .coordinate-rank {
200 |   top: 6px;
201 |   left: 7px;
202 | }
203 |
204 | .coordinate-file {
205 |   right: 7px;
206 |   bottom: 6px;
207 | }
208 |
209 | .control-panel,
210 | .info-panel {
211 |   min-width: 0;
212 |   min-height: 0;
213 |   max-height: var(--panel-max-height);
214 |   display: flex;
215 |   flex-direction: column;
216 |   gap: 14px;
217 |   overflow: auto;
218 |   overscroll-behavior: contain;
219 |   scrollbar-gutter: stable;
220 | }
221 |
222 | .control-panel {
223 |   justify-self: end;
224 | }
225 |
226 | .info-panel {
227 |   justify-self: start;
228 | }
229 |
230 | .control-label {
231 |   margin: 0;
232 |   color: var(--muted);
233 |   font-size: 0.78rem;
234 |   font-weight: 800;
235 |   letter-spacing: 0;
236 |   text-transform: uppercase;
237 | }
238 |
239 | .control-group,
240 | .status-panel,
241 | .ai-candidate-panel,
242 | .history-panel {
243 |   display: grid;
244 |   gap: 10px;
245 |   padding-top: 16px;
246 |   border-top: 1px solid var(--line);
247 | }
248 |
249 | .history-panel {

```

```

250 | min-height: 0;
251 | flex: 1;
252 | grid-template-rows: auto minmax(0, 1fr);
253 | }
254 |
255 | .segmented-control {
256 |   display: grid;
257 |   grid-template-columns: repeat(2, minmax(0, 1fr));
258 |   gap: 4px;
259 |   padding: 4px;
260 |   border: 1px solid var(--line);
261 |   border-radius: 8px;
262 |   background: var(--panel-soft);
263 | }
264 |
265 | .segmented-control-three {
266 |   grid-template-columns: repeat(3, minmax(0, 1fr));
267 | }
268 |
269 | .clock-preset-control {
270 |   display: grid;
271 |   grid-template-columns: repeat(5, minmax(0, 1fr));
272 |   gap: 4px;
273 |   padding: 4px;
274 |   border: 1px solid var(--line);
275 |   border-radius: 8px;
276 |   background: var(--panel-soft);
277 | }
278 |
279 | .segmented-button,
280 | .primary-button,
281 | .ghost-button,
282 | .promotion-button {
283 |   min-height: 42px;
284 |   border-radius: 6px;
285 |   font-weight: 800;
286 |   letter-spacing: 0;
287 | }
288 |
289 | .segmented-button {
290 |   border: 0;
291 |   background: var(--clear-color);
292 |   color: var(--muted);
293 | }
294 |
295 | .segmented-button.is-active {
296 |   background: var(--button);
297 |   color: var(--button-text);
298 | }
299 |
300 | .clock-settings .segmented-button {
301 |   min-height: 36px;
302 |   padding-inline: 4px;
303 |   font-size: 0.82rem;
304 | }
305 |
306 | .ai-settings {
307 |   gap: 12px;
308 | }
309 |
310 | .range-control,
311 | .toggle-control {
312 |   display: grid;
313 |   gap: 8px;
314 |   min-width: 0;
315 |   color: var(--ink);
316 |   font-size: 0.92rem;
317 |   font-weight: 800;
318 | }
319 |
320 | .range-control span,

```

```

321 | .toggle-control {
322 |   align-items: center;
323 | }
324 |
325 | .range-control span {
326 |   display: flex;
327 |   justify-content: space-between;
328 |   gap: 10px;
329 | }
330 |
331 | .range-control strong {
332 |   color: var(--accent);
333 |   white-space: nowrap;
334 | }
335 |
336 | .range-control input[type='range'] {
337 |   width: 100%;
338 |   accent-color: var(--button);
339 | }
340 |
341 | .toggle-control {
342 |   grid-template-columns: auto minmax(0, 1fr);
343 |   padding: 10px;
344 |   border: 1px solid var(--line);
345 |   border-radius: 8px;
346 |   background: var(--panel-raised);
347 | }
348 |
349 | .toggle-control input {
350 |   width: 18px;
351 |   height: 18px;
352 |   accent-color: var(--button);
353 | }
354 |
355 | .action-row {
356 |   display: grid;
357 |   grid-template-columns: 1fr 1fr;
358 |   gap: 10px;
359 | }
360 |
361 | .primary-button,
362 | .ghost-button {
363 |   border: 1px solid var(--button);
364 | }
365 |
366 | .primary-button {
367 |   background: var(--button);
368 |   color: var(--button-text);
369 | }
370 |
371 | .ghost-button {
372 |   background: var(--clear-color);
373 |   color: var(--button);
374 | }
375 |
376 | .primary-button:disabled,
377 | .ghost-button:disabled,
378 | .segmented-button:disabled {
379 |   opacity: 0.46;
380 | }
381 |
382 | .game-status {
383 |   min-height: 1.4em;
384 |   color: var(--accent);
385 |   font-size: 1.35rem;
386 |   line-height: 1.2;
387 | }
388 |
389 | .clock-display {
390 |   width: 100%;
391 |   height: 100%;

```

```

392 | min-height: 0;
393 | min-width: 0;
394 | display: flex;
395 | flex-direction: column;
396 | gap: 10px;
397 | justify-content: space-between;
398 | }
399 |
400 | .clock-card {
401 |   --clock-bg: var(--panel-raised);
402 |   --clock-border: var(--line);
403 |   --clock-side: var(--muted);
404 |   --clock-time: var(--ink);
405 |   width: 100%;
406 |   min-block-size: 82px;
407 |   max-block-size: 110px;
408 |   min-width: 0;
409 |   display: grid;
410 |   place-content: center;
411 |   gap: 4px;
412 |   padding: 12px 8px;
413 |   border: 1px solid var(--clock-border);
414 |   border-radius: 8px;
415 |   background: var(--clock-bg);
416 |   text-align: center;
417 | }
418 |
419 | #white-clock {
420 |   --clock-bg: oklch(99% 0.01 85deg / 1);
421 |   --clock-border: oklch(81% 0.02 80deg / 1);
422 |   --clock-side: oklch(54% 0.01 185deg / 1);
423 |   --clock-time: oklch(26% 0.01 250deg / 1);
424 | }
425 |
426 | #black-clock {
427 |   --clock-bg: oklch(33% 0.03 205deg / 1);
428 |   --clock-border: oklch(25% 0.02 210deg / 1);
429 |   --clock-side: oklch(99% 0.01 85deg / 0.74);
430 |   --clock-time: oklch(99% 0.01 85deg / 1);
431 | }
432 |
433 | .clock-card.is-active {
434 |   border-color: var(--button);
435 |   box-shadow:
436 |     inset 0 0 0 2px var(--active-ring),
437 |     0 0 0 2px var(--active-halo);
438 | }
439 |
440 | .clock-card.is-warning .clock-time {
441 |   color: var(--warning);
442 | }
443 |
444 | .clock-card.is-danger .clock-time,
445 | .clock-card.is-timeout .clock-time {
446 |   color: var(--capture);
447 | }
448 |
449 | .clock-side {
450 |   color: var(--clock-side);
451 |   font-size: 0.72rem;
452 |   font-weight: 800;
453 |   line-height: 1;
454 | }
455 |
456 | .clock-time {
457 |   font-variant-numeric: tabular-nums;
458 |   color: var(--clock-time);
459 |   font-size: 1.35rem;
460 |   line-height: 1;
461 | }
462 |

```

```

463 | .status-grid-{
464 |   display: grid;
465 |   grid-template-columns: 1fr;
466 |   gap: 8px;
467 |   margin: 0;
468 | }
469 |
470 | .status-grid-div-{
471 |   min-width: 0;
472 |   padding: 10px;
473 |   border: 1px solid var(--line);
474 |   border-radius: 8px;
475 |   background: var(--panel-raised);
476 | }
477 |
478 | .status-grid-dt,
479 | .status-grid-dd-{
480 |   margin: 0;
481 | }
482 |
483 | .status-grid-dt-{
484 |   color: var(--muted);
485 |   font-size: 0.72rem;
486 |   font-weight: 800;
487 | }
488 |
489 | .status-grid-dd-{
490 |   margin-top: 4px;
491 |   overflow-wrap: anywhere;
492 |   font-size: 0.92rem;
493 |   font-weight: 800;
494 | }
495 |
496 | .history-header-{
497 |   display: flex;
498 |   align-items: center;
499 |   justify-content: space-between;
500 | }
501 |
502 | .ai-candidate-panel-{
503 |   min-height: 0;
504 | }
505 |
506 | .ai-info-grid-{
507 |   display: grid;
508 |   grid-template-columns: repeat(2, minmax(0, 1fr));
509 |   gap: 6px;
510 |   margin: 0;
511 | }
512 |
513 | .ai-info-grid-div-{
514 |   min-width: 0;
515 |   padding: 8px;
516 |   border: 1px solid var(--line);
517 |   border-radius: 8px;
518 |   background: var(--panel-raised);
519 | }
520 |
521 | .ai-info-grid-dt,
522 | .ai-info-grid-dd-{
523 |   margin: 0;
524 | }
525 |
526 | .ai-info-grid-dt-{
527 |   color: var(--muted);
528 |   font-size: 0.64rem;
529 |   font-weight: 800;
530 |   text-transform: uppercase;
531 | }
532 |
533 | .ai-info-grid-dd-{

```

```

534 | margin-top: 3px;
535 | overflow: hidden;
536 | font-size: 0.8rem;
537 | font-weight: 800;
538 | line-height: 1.1;
539 | text-overflow: ellipsis;
540 | white-space: nowrap;
541 | }
542 |
543 | .ai-info-grid dd[id='ai-info-nodes'],
544 | .ai-info-grid dd[id='ai-info-score'],
545 | .ai-info-grid dd[id='ai-info-depth'] {
546 |   font-variant-numeric: tabular-nums;
547 | }
548 |
549 | .ai-candidate-header {
550 |   display: flex;
551 |   align-items: baseline;
552 |   justify-content: space-between;
553 |   gap: 8px;
554 | }
555 |
556 | .ai-candidate-meta {
557 |   min-width: 0;
558 |   overflow: hidden;
559 |   color: var(--muted);
560 |   font-size: 0.68rem;
561 |   font-weight: 800;
562 |   text-align: right;
563 |   text-overflow: ellipsis;
564 |   white-space: nowrap;
565 | }
566 |
567 | .ai-candidate-table {
568 |   min-height: 0;
569 |   overflow: hidden;
570 |   border: 1px solid var(--line);
571 |   border-radius: 8px;
572 |   background: var(--panel-raised);
573 | }
574 |
575 | .ai-candidate-list {
576 |   max-height: 172px;
577 |   overflow: auto;
578 | }
579 |
580 | .ai-candidate-row {
581 |   display: grid;
582 |   grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
583 |   gap: 6px;
584 |   align-items: center;
585 |   min-width: 0;
586 |   padding: 6px 8px;
587 |   border-bottom: 1px solid var(--line-soft);
588 |   font-size: 0.72rem;
589 |   font-weight: 800;
590 |   line-height: 1.15;
591 | }
592 |
593 | .ai-candidate-row:last-child {
594 |   border-bottom: 0;
595 | }
596 |
597 | .ai-candidate-head {
598 |   color: var(--muted);
599 |   background: var(--panel-strong);
600 |   font-size: 0.66rem;
601 |   text-transform: uppercase;
602 | }
603 |
604 | .ai-candidate-row.is-selected {

```

```

605 | background: var(--accent-soft);
606 | }
607 |
608 | .ai-candidate-row.is-principal:not(.is-selected){
609 | background: var(--button-soft);
610 | }
611 |
612 | .ai-candidate-san,
613 | .ai-candidate-score,
614 | .ai-candidate-weight,
615 | .ai-candidate-probability {
616 | min-width: 0;
617 | overflow: hidden;
618 | text-overflow: ellipsis;
619 | white-space: nowrap;
620 | }
621 |
622 | .ai-candidate-score,
623 | .ai-candidate-weight,
624 | .ai-candidate-probability {
625 | text-align: right;
626 | font-variant-numeric: tabular-nums;
627 | }
628 |
629 | .ai-candidate-badge {
630 | display: inline-block;
631 | margin-left: 4px;
632 | padding: 1px 3px;
633 | border-radius: 4px;
634 | background: var(--button);
635 | color: var(--button-text);
636 | font-size: 0.56rem;
637 | line-height: 1;
638 | vertical-align: 1px;
639 | }
640 |
641 | .move-list {
642 | min-height: 0;
643 | max-height: none;
644 | overflow: auto;
645 | border: 1px solid var(--line);
646 | border-radius: 8px;
647 | background: var(--panel-raised);
648 | }
649 |
650 | .move-row {
651 | display: grid;
652 | grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
653 | gap: 8px;
654 | padding: 8px 10px;
655 | border-bottom: 1px solid var(--line-firm);
656 | font-weight: 800;
657 | }
658 |
659 | .move-row:last-child {
660 | border-bottom: 0;
661 | }
662 |
663 | .move-number {
664 | color: var(--muted);
665 | }
666 |
667 | .move-san {
668 | min-width: 0;
669 | overflow-wrap: anywhere;
670 | }
671 |
672 | .move-empty {
673 | margin: 0;
674 | padding: 14px 12px;
675 | color: var(--muted);

```



```

676 |   font-weight: 700;
677 | }
678 |
679 | .promotion-overlay {
680 |   position: fixed;
681 |   inset: 0;
682 |   z-index: 10;
683 |   display: grid;
684 |   place-items: center;
685 |   padding: 20px;
686 |   background: var(--overlay);
687 | }
688 |
689 | .promotion-overlay[hidden] {
690 |   display: none;
691 | }
692 |
693 | .promotion-dialog {
694 |   width: 360px;
695 |   display: grid;
696 |   gap: 16px;
697 |   padding: 20px;
698 |   border: 1px solid var(--line);
699 |   border-radius: 8px;
700 |   background: var(--panel);
701 |   box-shadow: var(--shadow);
702 | }
703 |
704 | .promotion-dialog h2 {
705 |   margin: 0;
706 |   font-size: 1.1rem;
707 | }
708 |
709 | .promotion-choices {
710 |   display: grid;
711 |   grid-template-columns: repeat(4, minmax(0, 1fr));
712 |   gap: 8px;
713 | }
714 |
715 | .promotion-button {
716 |   min-height: 70px;
717 |   border: 1px solid var(--button);
718 |   background: var(--button-text);
719 |   color: var(--ink);
720 |   font-size: 2.4rem;
721 |   line-height: 1;
722 | }
723 |

```

==== src/styles/reset.css (560 B / 670e71371181) ====

```

1 | *,
2 | *::before,
3 | *::after {
4 |   box-sizing: border-box;
5 | }
6 |
7 | html {
8 |   min-width: 1280px;
9 |   min-height: 100%;
10 |   background: var(--page);
11 |   color-scheme: light;
12 | }
13 |
14 | body {
15 |   min-width: 1280px;
16 |   min-height: 100vh;
17 |   min-height: 100dvh;
18 |   margin: 0;
19 |   background: var(--page);
20 |   font-family:
21 |     Inter,

```

```
22 | | | |.ui-sans-serif,  
23 | | | |.system-ui,  
24 | | | |.apple-system,  
25 | | | |.BlinkMacSystemFont,  
26 | | | |.'Segoe UI',  
27 | | | |.sans-serif;  
28 | | | |text-rendering: geometricPrecision;  
29 | | }  
30 |  
31 | button {  
32 | | | |font: inherit;  
33 | | }  
34 |  
35 | button:not(:disabled) {  
36 | | | |cursor: pointer;  
37 | | }  
38 |  
39 | button:disabled {  
40 | | | |cursor: not-allowed;  
41 | | }  
42 |
```